Linelist Modules

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/ILinelistCodeTransferPlan.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1  'True
END
Attribute VB_Name = "ILinelistCodeTransferPlan"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Attribute VB_Description = "Describes the VBA components that must be transferred into the linelist workbook"

'@Interface
'@Folder("Linelist")
'@ModuleDescription("Describes the VBA components that must be transferred into the linelist workbook")
'@IgnoreModule UnrecognizedAnnotation, SuperfluousAnnotationArgument, UseMeaningfulName

Option Explicit

'@section Component collections
'==============================================================================
'@sub-title Returns the list of class modules that must be cloned into the target workbook.
Public Function ClassComponentNames() As BetterArray
End Function

'@sub-title Returns the list of standard modules that must be cloned into the target workbook.
Public Function ModuleComponentNames() As BetterArray
End Function

'@sub-title Returns the userform names that must exist in the target workbook.
Public Function FormNames() As BetterArray
End Function

'@section Workbook module metadata
'==============================================================================
'@sub-title Name of the workbook module whose code must be synchronised.
Public Property Get WorkbookModuleName() As String
End Property
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/ILinelistCodeTransferService.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1  'True
END
Attribute VB_Name = "ILinelistCodeTransferService"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Attribute VB_Description = "Facade coordinating Linelist code module and form transfers"

'@Interface
'@Folder("Linelist")
'@ModuleDescription("Facade coordinating Linelist code module and form transfers")
'@IgnoreModule UnrecognizedAnnotation, SuperfluousAnnotationArgument, UseMeaningfulName

Option Explicit

Public Sub TransferComponent(ByVal componentName As String, _
                             ByVal scope As LinelistCodeScope, _
                             ByVal source As Workbook, _
                             ByVal target As Workbook)
End Sub

Public Sub TransferWorkbookModule(ByVal moduleName As String, _
                                  ByVal source As Workbook, _
                                  ByVal target As Workbook)
End Sub

Public Sub TransferForm(ByVal formName As String, _
                        ByVal source As Workbook, _
                        ByVal target As Workbook)
End Sub

Public Sub TransferComponents(ByVal componentNames As BetterArray, _
                              ByVal scope As LinelistCodeScope, _
                              ByVal source As Workbook, _
                              ByVal target As Workbook)
End Sub

Public Property Get StrategyName() As String
End Property
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/ILinelistCodeTransferStrategy.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1  'True
END
Attribute VB_Name = "ILinelistCodeTransferStrategy"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Attribute VB_Description = "Transfers VBA components between workbooks using a specific mechanism"

'@Interface
'@Folder("Linelist")
'@ModuleDescription("Transfers VBA components between workbooks using a specific mechanism")
'@IgnoreModule UnrecognizedAnnotation, SuperfluousAnnotationArgument, UseMeaningfulName

Option Explicit

'@section Component transfer
'==============================================================================
Public Sub TransferComponent(ByVal componentName As String, _
                             ByVal scope As LinelistCodeScope, _
                             ByVal source As Workbook, _
                             ByVal target As Workbook)
End Sub

Public Sub TransferWorkbookModule(ByVal moduleName As String, _
                                  ByVal source As Workbook, _
                                  ByVal target As Workbook)
End Sub

Public Sub TransferForm(ByVal formName As String, _
                        ByVal source As Workbook, _
                        ByVal target As Workbook)
End Sub


'@section Diagnostics
'==============================================================================
Public Property Get Name() As String
End Property
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/ILinelistLifecycleManager.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1  'True
END
Attribute VB_Name = "ILinelistLifecycleManager"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Attribute VB_Description = "Coordinates reset and disposal operations for the linelist output workbook"

'@Interface
'@Folder("Linelist")
'@ModuleDescription("Coordinates reset and disposal operations for the linelist output workbook")
'@IgnoreModule UnrecognizedAnnotation, UseMeaningfulName, SuperfluousAnnotationArgument

Option Explicit

'@section Lifecycle operations
'==============================================================================
'@sub-title Clears workbook state and temporary artefacts so a new preparation run can begin.
Public Sub Reset(Optional ByVal saveChanges As Boolean = False)
End Sub

'@sub-title Performs a terminal shutdown, releasing resources and clearing references.
Public Sub Dispose(Optional ByVal saveChanges As Boolean = False)
End Sub
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/ILinelistPreparationContext.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1  'True
END
Attribute VB_Name = "ILinelistPreparationContext"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Attribute VB_Description = "Runtime context shared across Linelist preparation steps"

'@Interface
'@Folder("Linelist")
'@ModuleDescription("Runtime context shared across Linelist preparation steps")
'@IgnoreModule UnrecognizedAnnotation, SuperfluousAnnotationArgument, UseMeaningfulName

Option Explicit

'@section Core dependencies
'==============================================================================
Public Property Get Linelist() As ILinelist
End Property

Public Property Get Specifications() As ILinelistSpecs
End Property

Public Property Get Dictionary() As ILLdictionary
End Property

Public Property Get SheetFormatter() As ILinelistSheetNameFormatter
End Property

Public Property Get StateScope() As IApplicationState
End Property
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/ILinelistPreparationPipeline.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1  'True
END
Attribute VB_Name = "ILinelistPreparationPipeline"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Attribute VB_Description = "Coordinates the multi-step Linelist preparation flow"

'@Interface
'@Folder("Linelist")
'@ModuleDescription("Coordinates the multi-step Linelist preparation flow")
'@IgnoreModule UnrecognizedAnnotation, SuperfluousAnnotationArgument, UseMeaningfulName

Option Explicit

'@section Preparation entry point
'==============================================================================
Public Sub Prepare()
End Sub
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/ILinelistPreparationStep.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1  'True
END
Attribute VB_Name = "ILinelistPreparationStep"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Attribute VB_Description = "Single stage of the Linelist preparation pipeline"

'@Interface
'@Folder("Linelist")
'@ModuleDescription("Single stage of the Linelist preparation pipeline")
'@IgnoreModule UnrecognizedAnnotation, SuperfluousAnnotationArgument, UseMeaningfulName

Option Explicit

'@section Execution
'==============================================================================
Public Sub Execute(ByVal context As ILinelistPreparationContext)
End Sub
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/ILinelistSaveWorkflow.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1  'True
END
Attribute VB_Name = "ILinelistSaveWorkflow"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Attribute VB_Description = "Coordinates the final save sequence for the Linelist workbook"

'@Interface
'@Folder("Linelist")
'@ModuleDescription("Coordinates the final save sequence for the Linelist workbook")
'@IgnoreModule UnrecognizedAnnotation, UseMeaningfulName, SuperfluousAnnotationArgument

Option Explicit

'@section Save operations
'==============================================================================
'@sub-title Executes the save process using the supplied context.
Public Sub Save(ByVal context As ILinelistPreparationContext)
End Sub
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/ILinelistSheetNameFormatter.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1  'True
END
Attribute VB_Name = "ILinelistSheetNameFormatter"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Attribute VB_Description = "Centralises sheet-name construction logic for the Linelist workflow"

'@Interface
'@Folder("Linelist")
'@ModuleDescription("Centralises sheet-name construction logic for the Linelist workflow")
'@IgnoreModule UnrecognizedAnnotation, SuperfluousAnnotationArgument, UseMeaningfulName

Option Explicit

'@section Scope enumerations
'==============================================================================
Public Enum LinelistSheetScope
    sheetScopeStandard = 1
    sheetScopePrint = 2
    sheetScopeCrf = 3
End Enum

Public Enum LinelistCodeScope
    codeScopeClass = 1
    codeScopeModule = 2
End Enum


'@section Configuration
'==============================================================================
'@sub-title Prefix assigned to printed worksheets.
Public Property Get PrintPrefix() As String
End Property

'@sub-title Prefix assigned to CRF worksheets.
Public Property Get CrfPrefix() As String
End Property

'@sub-title Maximum supported sheet-name length including prefixes.
Public Property Get SheetNameMaxLength() As Long
End Property


'@section Sheet helpers
'==============================================================================
'@sub-title Resolve the scoped sheet name within the allowed length budget.
'@param baseName Candidate workbook sheet name.
'@param scope Sheet naming scope (standard, print, crf).
'@return Sheet name trimmed to the configured maximum with the scope prefix applied.
Public Function FormatSheetName(ByVal baseName As String, _
                                Optional ByVal scope As LinelistSheetScope = sheetScopeStandard) As String
End Function

'@sub-title Check whether a scoped sheet exists in the workbook.
Public Function SheetExists(ByVal workbook As Workbook, ByVal baseName As String, _
                            Optional ByVal scope As LinelistSheetScope = sheetScopeStandard) As Boolean
End Function

'@sub-title Retrieve a scoped worksheet or raise if not found.
Public Function ResolveWorksheet(ByVal workbook As Workbook, ByVal baseName As String, _
                                 Optional ByVal scope As LinelistSheetScope = sheetScopeStandard) As Worksheet
End Function
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/ILinelistSheetNavigator.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1  'True
END
Attribute VB_Name = "ILinelistSheetNavigator"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Attribute VB_Description = "Provides safe access and activation helpers for linelist worksheets"

'@Interface
'@Folder("Linelist")
'@ModuleDescription("Provides safe access and activation helpers for linelist worksheets")
'@IgnoreModule UnrecognizedAnnotation, UseMeaningfulName, SuperfluousAnnotationArgument

Option Explicit

'@section Core navigation
'==============================================================================
'@sub-title Activate the admin worksheet using cached translation metadata.
Public Sub ActivateAdminSheet()
End Sub

'@sub-title Activate the instruction worksheet using cached translation metadata.
Public Sub ActivateInstructionSheet()
End Sub

'@sub-title Activate a translated sheet by base name and scope.
Public Function ActivateSheet(ByVal baseName As String, _
                              Optional ByVal scope As LinelistSheetScope = sheetScopeStandard) As Worksheet
End Function

'@sub-title Resolve a worksheet by translated base name without activation.
Public Function ResolveSheet(ByVal baseName As String, _
                             Optional ByVal scope As LinelistSheetScope = sheetScopeStandard) As Worksheet
End Function

'@sub-title Check whether a translated sheet exists for the provided scope.
Public Function SheetExists(ByVal baseName As String, _
                            Optional ByVal scope As LinelistSheetScope = sheetScopeStandard) As Boolean
End Function

'@section Diagnostics
'==============================================================================
'@sub-title Returns the last sheet name activated by the navigator (scoped name).
Public Property Get LastActivatedSheetName() As String
End Property

'@sub-title Returns the number of cached translation lookups performed.
Public Property Get TranslationCacheHits() As Long
End Property
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/ILinelistSpecs.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1  'True
END
Attribute VB_Name = "ILinelistSpecs"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False


'@Folder("Linelist")
'@ModuleDescription("Contract describing reusable linelist specifications accessors")
'@IgnoreModule UnrecognizedAnnotation, UseMeaningfulName, HungarianNotation
'@InterfaceDescription("Surface area consumed by linelist writers and contexts to retrieve workbook driven specifications")

Option Explicit

'@section Lifecycle
'==============================================================================
'@description Refresh any cached collaborators so subsequent reads query the workbook again.
Public Sub ResetCaches()
End Sub

'@description Workbook hosting the specifications.
Public Property Get Workbook() As Workbook
End Property
```

```
28
29  '@section Core Specifications
30  '==============================================================================
31  Public Property Get Dictionary() As ILLdictionary
32  End Property
33
34  Public Property Get Choices() As ILLChoices
35  End Property
36
37  Public Property Get DesignFormat() As ILLFormat
38  End Property
39
40  Public Property Get Analysis() As ILLAnalysis
41  End Property
42
43  Public Property Get ExportObject() As ILLExport
44  End Property
45
46  Public Property Get Password() As IPasswords
47  End Property
48
49  Public Property Get GeoObject() As ILLGeo
50  End Property
51
52  Public Property Get FormulaDataObject() As IFormulaData
53  End Property
54
55  Public Property Get MainObject() As IMain
56  End Property
57
58  Public Property Get Translations() As ILLTranslations
59  End Property
60
61  Public Property Get DesignerTranslations() As IDesTranslation
62  End Property
63
64  Public Property Get TransObject(Optional ByVal translationScope As Byte = TranslationOfMessages) As ITranslationObject
65  End Property
66
67  '@section Metadata Helpers
68  '==============================================================================
69  Public Property Get TemporarySheetName(ByVal scope As Byte) As String
70  End Property
71
72  Public Property Get Value(ByVal tagName As String) As String
73  End Property
74
75  Public Function Categories(ByVal varName As String, Optional ByVal useShortlabels As Boolean = False) As BetterArray
76  End Function
77
78  '@section Workbook Management
79  '==============================================================================
80  Public Sub Prepare()
81  End Sub
82
83  Public Sub Export(ByVal toWkb As Workbook)
84  End Sub
85
86  Public Sub ErrorManage(Optional ByVal textMessage As String = vbNullString)
87  End Sub
88
89  '@section Writer Factories
90  '==============================================================================
91  Public Function CreateSectionContext(ByVal linelist As ILinelist, ByVal startRow As Long) As ILLSectionContext
92  End Function
93
94  Public Function CreateSectionWriter() As ILLSectionWriter
95  End Function
96
97  Public Function CreateSubSectionWriter() As ILLSubSectionWriter
98  End Function
99
100 Public Function CreateVarContext(ByVal linelist As ILinelist, ByVal variableName As String) As ILLVarContext
101 End Function
102
103 Public Function CreateVarWriter() As ILLVarWriter
104 End Function
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/ILinelistWorkbookAccessor.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1  'True
END
Attribute VB_Name = "ILinelistWorkbookAccessor"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Attribute VB_Description = "Exposes the workbook currently being assembled by the linelist"

'@Interface
'@Folder("Linelist")
'@ModuleDescription("Exposes the workbook currently being assembled by the linelist")
'@IgnoreModule UnrecognizedAnnotation, UseMeaningfulName, SuperfluousAnnotationArgument

Option Explicit

'@section Workbook access
'==============================================================================
'@sub-title Returns the in-flight output workbook used by the linelist builder.
Public Property Get OutputWorkbook() As Workbook
End Property
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/ILinelistWorkbookReleaser.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1  'True
END
Attribute VB_Name = "ILinelistWorkbookReleaser"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Attribute VB_Description = "Allows lifecycle helpers to clear linelist workbook references once they are closed"

'@Interface
'@Folder("Linelist")
'@ModuleDescription("Allows lifecycle helpers to clear linelist workbook references once they are closed")
'@IgnoreModule UnrecognizedAnnotation, UseMeaningfulName, SuperfluousAnnotationArgument

Option Explicit

'@section Workbook release
'==============================================================================
'@sub-title Clears the cached workbook instance so a fresh output workbook can be created afterwards.
Public Sub ClearOutputWorkbook()
End Sub
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/LinelistCodeTransferService.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1  'True
END
Attribute VB_Name = "LinelistCodeTransferService"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Attribute VB_Description = "Coordinates VBA component transfers using the appropriate strategy for the host"

'@Folder("Linelist")
'@ModuleDescription("Coordinates VBA component transfers using the appropriate strategy for the host")
'@IgnoreModule UnrecognizedAnnotation, UseMeaningfulName, ProcedureNotUsed
'@depends ProjectError

Option Explicit

Implements ILinelistCodeTransferService

'@section State and constants
'==============================================================================
Private Type TCodeTransferService
    strategy As ILinelistCodeTransferStrategy
End Type
```

```
26
27  Private this As TCodeTransferService
28  Private Const CLASSNAME As String = "LinelistCodeTransferService"
29
30
31  '@section Factory helpers
32  '===============================================================================
33  Public Function Create(Optional ByVal strategy As ILinelistCodeTransferStrategy) As ILinelistCodeTransferService
34     Dim service As LinelistCodeTransferService
35     Set service = New LinelistCodeTransferService
36     service.Initialise strategy
37     Set Create = service
38  End Function
39
40  Friend Sub Initialise(Optional ByVal strategy As ILinelistCodeTransferStrategy)
41     If strategy Is Nothing Then
42        Set this.strategy = DetermineStrategy
43     Else
44        Set this.strategy = strategy
45     End If
46  End Sub
47
48
49  '@section Public API
50  '===============================================================================
51  Public Sub TransferComponent(ByVal componentName As String, _
52                               ByVal scope As LinelistCodeScope, _
53                               ByVal source As Workbook, _
54                               ByVal target As Workbook)
55     ValidateArguments componentName, source, target
56     this.strategy.TransferComponent componentName, scope, source, target
57  End Sub
58
59  Public Sub TransferWorkbookModule(ByVal moduleName As String, _
60                                    ByVal source As Workbook, _
61                                    ByVal target As Workbook)
62     ValidateArguments moduleName, source, target
63     this.strategy.TransferWorkbookModule moduleName, source, target
64  End Sub
65
66  Public Sub TransferForm(ByVal formName As String, _
67                          ByVal source As Workbook, _
68                          ByVal target As Workbook)
69     ValidateArguments formName, source, target
70     this.strategy.TransferForm formName, source, target
71  End Sub
72
73  Public Sub TransferComponents(ByVal componentNames As BetterArray, _
74                                ByVal scope As LinelistCodeScope, _
75                                ByVal source As Workbook, _
76                                ByVal target As Workbook)
77     Dim index As Long
78
79     If componentNames Is Nothing Then
80        ThrowError ProjectError.ObjectNotInitialized, "Component list cannot be Nothing"
81     End If
82
83     For index = componentNames.LowerBound To componentNames.UpperBound
84        TransferComponent CStr(componentNames.Item(index)), scope, source, target
85     Next index
86  End Sub
87
88  Public Property Get StrategyName() As String
89     If this.strategy Is Nothing Then
90        StrategyName = "(none)"
91     Else
92        StrategyName = this.strategy.Name
93     End If
94  End Property
95
96
97  '@section Internal helpers
98  '===============================================================================
99  Private Sub ValidateArguments(ByVal componentName As String, _
100                                ByVal source As Workbook, _
101                                ByVal target As Workbook)
102    If LenB(Trim$(componentName)) = 0 Then
103       ThrowError ProjectError.InvalidArgument, "Component name cannot be empty"
104    End If
105
106    EnsureWorkbook source, "Source workbook cannot be Nothing"
107    EnsureWorkbook target, "Target workbook cannot be Nothing"
108 End Sub
109
110 Private Sub EnsureWorkbook(ByVal candidate As Workbook, ByVal message As String)
111    If candidate Is Nothing Then
112       ThrowError ProjectError.ObjectNotInitialized, message
```

```
113     End If
114
115     If StrComp(TypeName(candidate), "Workbook", vbTextCompare) <> 0 Then
116         ThrowError ProjectError.InvalidArgument, "Workbook argument must be an Excel workbook"
117     End If
118 End Sub
119
120 Private Function DetermineStrategy() As ILinelistCodeTransferStrategy
121     If CanAccessVBIDE() Then
122         Set DetermineStrategy = LinelistVBIDECodeTransferStrategy.Create
123     Else
124         Set DetermineStrategy = LinelistFallbackCodeTransferStrategy.Create
125     End If
126 End Function
127
128 Private Function CanAccessVBIDE() As Boolean
129     On Error GoTo NoAccess
130         Dim vbeObject As Object
131         Set vbeObject = Application.VBE
132         CanAccessVBIDE = Not vbeObject Is Nothing
133     On Error GoTo 0
134     Exit Function
135 NoAccess:
136     Err.Clear
137     CanAccessVBIDE = False
138 End Function
139
140 Private Sub ThrowError(ByVal errNumber As ProjectError, ByVal message As String)
141     Err.Raise errNumber, CLASSNAME, message
142 End Sub
143
144
145 '@section Interface implementation
146 '=============================================================================
147 Private Sub ILinelistCodeTransferService_TransferComponent(ByVal componentName As String, _
148                                                           ByVal scope As LinelistCodeScope, _
149                                                           ByVal source As Workbook, _
150                                                           ByVal target As Workbook)
151     TransferComponent componentName, scope, source, target
152 End Sub
153
154 Private Sub ILinelistCodeTransferService_TransferWorkbookModule(ByVal moduleName As String, _
155                                                                ByVal source As Workbook, _
156                                                                ByVal target As Workbook)
157     TransferWorkbookModule moduleName, source, target
158 End Sub
159
160 Private Sub ILinelistCodeTransferService_TransferForm(ByVal formName As String, _
161                                                       ByVal source As Workbook, _
162                                                       ByVal target As Workbook)
163     TransferForm formName, source, target
164 End Sub
165
166 Private Sub ILinelistCodeTransferService_TransferComponents(ByVal componentNames As BetterArray, _
167                                                             ByVal scope As LinelistCodeScope, _
168                                                             ByVal source As Workbook, _
169                                                             ByVal target As Workbook)
170     TransferComponents componentNames, scope, source, target
171 End Sub
172
173 Private Property Get ILinelistCodeTransferService_StrategyName() As String
174     ILinelistCodeTransferService_StrategyName = StrategyName
175 End Property
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/LinelistCodeTransferStep.cls

```
1   VERSION 1.0 CLASS
2   BEGIN
3     MultiUse = -1  'True
4   END
5   Attribute VB_Name = "LinelistCodeTransferStep"
6   Attribute VB_GlobalNameSpace = False
7   Attribute VB_Creatable = False
8   Attribute VB_PredeclaredId = False
9   Attribute VB_Exposed = False
10  Attribute VB_Description = "Preparation step that synchronises VBA components into the output linelist workbook"
11
12  '@Folder("Linelist")
13  '@ModuleDescription("Preparation step that synchronises VBA components into the output linelist workbook")
14  '@IgnoreModule UnrecognizedAnnotation, UseMeaningfulName, ProcedureNotUsed
15  '@depends ProjectError
16
17  Option Explicit
```

```
18
19 Implements ILinelistPreparationStep
20
21 '@section State and constants
22 '==============================================================================
23 Private Type TStep
24     service As ILinelistCodeTransferService
25     plan As ILinelistCodeTransferPlan
26 End Type
27
28 Private this As TStep
29 Private Const CLASSNAME As String = "LinelistCodeTransferStep"
30
31
32 '@section Factory helpers
33 '==============================================================================
34 Public Function Create(ByVal service As ILinelistCodeTransferService, _
35                       Optional ByVal plan As ILinelistCodeTransferPlan) As ILinelistPreparationStep
36
37     Dim stepObject As LinelistCodeTransferStep
38     Set stepObject = New LinelistCodeTransferStep
39     stepObject.Initialise service, plan
40     Set Create = stepObject
41 End Function
42
43 Friend Sub Initialise(ByVal service As ILinelistCodeTransferService, _
44                      ByVal plan As ILinelistCodeTransferPlan)
45
46     EnsureService service
47
48     If plan Is Nothing Then
49         Set this.plan = LinelistDefaultCodeTransferPlan.Create
50     Else
51         Set this.plan = plan
52     End If
53
54     Set this.service = service
55 End Sub
56
57
58 '@section Execution helpers
59 '==============================================================================
60 Private Sub ExecuteTransfer(ByVal context As ILinelistPreparationContext)
61     Dim sourceWorkbook As Workbook
62     Dim targetWorkbook As Workbook
63
64     Set sourceWorkbook = ResolveSourceWorkbook(context)
65     Set targetWorkbook = ResolveTargetWorkbook(context)
66
67     TransferClassModules sourceWorkbook, targetWorkbook
68     TransferStandardModules sourceWorkbook, targetWorkbook
69     TransferForms sourceWorkbook, targetWorkbook
70     TransferWorkbookModule sourceWorkbook, targetWorkbook
71 End Sub
72
73 Private Sub TransferClassModules(ByVal sourceWorkbook As Workbook, ByVal targetWorkbook As Workbook)
74     Dim components As BetterArray
75
76     Set components = this.plan.ClassComponentNames
77     If components.Length = 0 Then Exit Sub
78
79     this.service.TransferComponents components, codeScopeClass, sourceWorkbook, targetWorkbook
80 End Sub
81
82 Private Sub TransferStandardModules(ByVal sourceWorkbook As Workbook, ByVal targetWorkbook As Workbook)
83     Dim modules As BetterArray
84
85     Set modules = this.plan.ModuleComponentNames
86     If modules.Length = 0 Then Exit Sub
87
88     this.service.TransferComponents modules, codeScopeModule, sourceWorkbook, targetWorkbook
89 End Sub
90
91 Private Sub TransferForms(ByVal sourceWorkbook As Workbook, ByVal targetWorkbook As Workbook)
92     Dim forms As BetterArray
93     Dim index As Long
94
95     Set forms = this.plan.FormNames
96     If forms.Length = 0 Then Exit Sub
97
98     For index = forms.LowerBound To forms.UpperBound
99         this.service.TransferForm CStr(forms.Item(index)), sourceWorkbook, targetWorkbook
100    Next index
101 End Sub
102
103 Private Sub TransferWorkbookModule(ByVal sourceWorkbook As Workbook, ByVal targetWorkbook As Workbook)
104    Dim moduleName As String
```

```
105
106     moduleName = Trim$(this.plan.WorkbookModuleName)
107     If LenB(moduleName) = 0 Then Exit Sub
108
109     this.service.TransferWorkbookModule moduleName, sourceWorkbook, targetWorkbook
110 End Sub
111
112
113 '@section Resolution helpers
114 '==============================================================================
115 Private Function ResolveSourceWorkbook(ByVal context As ILinelistPreparationContext) As Workbook
116     Dim specs As ILinelistSpecs
117     Dim workbook As Workbook
118
119     Set specs = context.Specifications
120     If specs Is Nothing Then
121         ThrowError ProjectError.ObjectNotInitialized, "Preparation context does not expose specifications"
122     End If
123
124     Set workbook = specs.Workbook
125     If workbook Is Nothing Then
126         ThrowError ProjectError.ObjectNotInitialized, "Specifications workbook reference cannot be Nothing"
127     End If
128
129     Set ResolveSourceWorkbook = workbook
130 End Function
131
132 Private Function ResolveTargetWorkbook(ByVal context As ILinelistPreparationContext) As Workbook
133     Dim linelist As ILinelist
134     Dim accessor As ILinelistWorkbookAccessor
135     Dim workbook As Workbook
136
137     Set linelist = context.Linelist
138     If linelist Is Nothing Then
139         ThrowError ProjectError.ObjectNotInitialized, "Preparation context must expose a linelist reference"
140     End If
141
142     Set accessor = CastWorkbookAccessor(linelist)
143     Set workbook = accessor.OutputWorkbook
144
145     If workbook Is Nothing Then
146         ThrowError ProjectError.ObjectNotInitialized, "Linelist output workbook cannot be Nothing"
147     End If
148
149     Set ResolveTargetWorkbook = workbook
150 End Function
151
152 Private Function CastWorkbookAccessor(ByVal candidate As ILinelist) As ILinelistWorkbookAccessor
153     Dim accessor As ILinelistWorkbookAccessor
154
155     On Error GoTo MissingInterface
156         Set accessor = candidate
157         On Error GoTo 0
158         Set CastWorkbookAccessor = accessor
159         Exit Function
160
161 MissingInterface:
162     Err.Clear
163     ThrowError ProjectError.InvalidOperation, _
164             "Linelist instance " & TypeName(candidate) & " does not expose ILinelistWorkbookAccessor"
165 End Function
166
167
168 '@section Validation helpers
169 '==============================================================================
170 Private Sub EnsureService(ByVal service As ILinelistCodeTransferService)
171     If service Is Nothing Then
172         ThrowError ProjectError.ObjectNotInitialized, "Code transfer service cannot be Nothing"
173     End If
174 End Sub
175
176 Private Sub ThrowError(ByVal errNumber As ProjectError, ByVal message As String)
177     Err.Raise errNumber, CLASSNAME, message
178 End Sub
179
180
181 '@section Interface implementation
182 '==============================================================================
183 Private Sub ILinelistPreparationStep_Execute(ByVal context As ILinelistPreparationContext)
184     If context Is Nothing Then
185         ThrowError ProjectError.ObjectNotInitialized, "Preparation context cannot be Nothing"
186     End If
187
188     ExecuteTransfer context
189 End Sub
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/LinelistDefaultCodeTransferPlan.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1  'True
END
Attribute VB_Name = "LinelistDefaultCodeTransferPlan"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Attribute VB_Description = "Provides the canonical list of VBA components required by the Linelist workbook"

'@Folder("Linelist")
'@ModuleDescription("Provides the canonical list of VBA components required by the Linelist workbook")
'@IgnoreModule UnrecognizedAnnotation, UseMeaningfulName, ProcedureNotUsed
'@depends ProjectError

Option Explicit

Implements ILinelistCodeTransferPlan

'@section State and constants
'==============================================================================
Private Type TPlan
    classModules As BetterArray
    standardModules As BetterArray
    formNames As BetterArray
    workbookModuleName As String
End Type

Private this As TPlan
Private Const CLASSNAME As String = "LinelistDefaultCodeTransferPlan"
Private Const DEFAULT_WORKBOOK_MODULE As String = "LinelistWorkbook"


'@section Factory helpers
'==============================================================================
Public Function Create() As ILinelistCodeTransferPlan
    Dim plan As LinelistDefaultCodeTransferPlan
    Set plan = New LinelistDefaultCodeTransferPlan
    plan.Initialise
    Set Create = plan
End Function

Friend Sub Initialise()
    Set this.classModules = BuildArray(DefaultClassModules())
    Set this.standardModules = BuildArray(DefaultStandardModules())
    Set this.formNames = BuildArray(DefaultFormNames())
    this.workbookModuleName = DEFAULT_WORKBOOK_MODULE
End Sub


'@section Defaults
'==============================================================================
Private Function DefaultClassModules() As Variant
    DefaultClassModules = Array( _
        "BetterArray", "DataSheet", "IDataSheet", "LLExport", _
        "ILLExport", "LLChoices", "ILLChoices", _
        "LLdictionary", "ILLdictionary", "LLGeo", _
        "ILLGeo", "Passwords", "IPasswords", _
        "LLSheets", "ILLSheets", "LLTranslations", _
        "ILLTranslations", "OSFiles", "IOSFiles", _
        "TranslationObject", "ITranslationObject", "LLVariables", _
        "ILLVariables", "ILLSpatial", "LLSpatial", _
        "DropdownLists", "IDropdownLists", "LLShowHide", _
        "ILLShowHide", "CustomTable", "ICustomTable", _
        "OutputSpecs", "IOutputSpecs", _
        "ImpRep", "IImpRep", "ImpSpecs", "IImpSpecs", _
        "UpVal", "IUpVal", "IGraphs", "Graphs", "AnaTabIds", _
        "IAnaTabIds", "ExportButtons", "IExportButtons" _
    )
End Function

Private Function DefaultStandardModules() As Variant
    DefaultStandardModules = Array( _
        "LinelistCustomFunctions", _
        "LinelistGeo", "LinelistEvents", _
        "EventsSpatialAnalysis", _
        "EventsLinelistButtons", "EventsGlobalAnalysis" _
    )
End Function

Private Function DefaultFormNames() As Variant
```

```
83      DefaultFormNames = Array( _
84          "F_Advanced", "F_Export", "F_ExportMig", _
85          "F_Geo", "F_ImportRep", _
86          "F_ShowHideLL", "F_ShowHidePrint", "F_ShowVarLabels", _
87          "F_EpiWeek" _
88      )
89  End Function
90
91
92  '@section Helpers
93  '===============================================================================
94  Private Function BuildArray(ByVal values As Variant) As BetterArray
95      Dim result As BetterArray
96      Dim index As Long
97
98      Set result = New BetterArray
99      result.LowerBound = 1
100
101     For index = LBound(values) To UBound(values)
102         result.Push CStr(values(index))
103     Next index
104
105     Set BuildArray = result
106 End Function
107
108 Private Sub ThrowError(ByVal errNumber As ProjectError, ByVal message As String)
109     Err.Raise errNumber, CLASSNAME, message
110 End Sub
111
112 Private Sub EnsureArray(ByVal candidate As BetterArray, ByVal description As String)
113     If candidate Is Nothing Then
114         ThrowError ProjectError.ObjectNotInitialized, description & " cannot be Nothing"
115     End If
116 End Sub
117
118
119 '@section Interface implementation
120 '===============================================================================
121 Private Function ILinelistCodeTransferPlan_ClassComponentNames() As BetterArray
122     EnsureArray this.classModules, "Class component list"
123     Set ILinelistCodeTransferPlan_ClassComponentNames = this.classModules
124 End Function
125
126 Private Function ILinelistCodeTransferPlan_ModuleComponentNames() As BetterArray
127     EnsureArray this.standardModules, "Standard module list"
128     Set ILinelistCodeTransferPlan_ModuleComponentNames = this.standardModules
129 End Function
130
131 Private Function ILinelistCodeTransferPlan_FormNames() As BetterArray
132     EnsureArray this.formNames, "Form list"
133     Set ILinelistCodeTransferPlan_FormNames = this.formNames
134 End Function
135
136 Private Property Get ILinelistCodeTransferPlan_WorkbookModuleName() As String
137     ILinelistCodeTransferPlan_WorkbookModuleName = this.workbookModuleName
138 End Property
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/LinelistFallbackCodeTransferStrategy.cls

```
1   VERSION 1.0 CLASS
2   BEGIN
3     MultiUse = -1  'True
4   END
5   Attribute VB_Name = "LinelistFallbackCodeTransferStrategy"
6   Attribute VB_GlobalNameSpace = False
7   Attribute VB_Creatable = False
8   Attribute VB_PredeclaredId = False
9   Attribute VB_Exposed = False
10  Attribute VB_Description = "Fallback strategy that reuses existing workbook components without VBIDE import/export"
11
12  '@Folder("Linelist")
13  '@ModuleDescription("Fallback strategy that reuses existing workbook components without VBIDE import/export")
14  '@IgnoreModule UnrecognizedAnnotation, UseMeaningfulName, ProcedureNotUsed
15  '@depends ProjectError
16
17  Option Explicit
18
19  Implements ILinelistCodeTransferStrategy
20
21  Private Const CLASSNAME As String = "LinelistFallbackCodeTransferStrategy"
22
23
24  '@section Factory helpers
```

```
25  '==============================================================================
26  Public Function Create() As ILinelistCodeTransferStrategy
27     Dim strategy As LinelistFallbackCodeTransferStrategy
28     Set strategy = New LinelistFallbackCodeTransferStrategy
29     Set Create = strategy
30  End Function
31
32
33  '@section Strategy contract
34  '==============================================================================
35  Public Sub TransferComponent(ByVal componentName As String, _
36                               ByVal scope As LinelistCodeScope, _
37                               ByVal source As Workbook, _
38                               ByVal target As Workbook)
39     Dim sourceComponent As Object
40     Dim targetComponent As Object
41     Dim codeModule As Object
42     Dim moduleText As String
43
44     Set sourceComponent = ResolveComponent(source, componentName)
45     Set targetComponent = EnsureTargetComponent(target, componentName)
46
47     Set codeModule = sourceComponent.CodeModule
48     moduleText = codeModule.Lines(1, codeModule.CountOfLines)
49
50     With targetComponent.CodeModule
51        If .CountOfLines > 0 Then .DeleteLines 1, .CountOfLines
52        If Len(moduleText) > 0 Then .AddFromString moduleText
53     End With
54  End Sub
55
56  Public Sub TransferWorkbookModule(ByVal moduleName As String, _
57                                    ByVal source As Workbook, _
58                                    ByVal target As Workbook)
59     Dim sourceModule As Object
60     Dim targetModule As Object
61     Dim moduleText As String
62
63     Set sourceModule = ResolveComponent(source, moduleName).CodeModule
64     Set targetModule = ResolveComponent(target, moduleName).CodeModule
65
66     moduleText = sourceModule.Lines(1, sourceModule.CountOfLines)
67
68     With targetModule
69        If .CountOfLines > 0 Then .DeleteLines 1, .CountOfLines
70        If Len(moduleText) > 0 Then .AddFromString moduleText
71     End With
72  End Sub
73
74  Public Sub TransferForm(ByVal formName As String, _
75                          ByVal source As Workbook, _
76                          ByVal target As Workbook)
77     'Fallback strategy assumes forms already exist in the target workbook.
78     If Not ComponentExists(target, formName) Then
79        ThrowError ProjectError.ElementNotFound, _
80                 "Form '" & formName & "' must be preloaded in the target workbook when VBIDE automation is unavailable"
81     End If
82  End Sub
83
84  Public Property Get Name() As String
85     Name = "Fallback"
86  End Property
87
88
89  '@section Helpers
90  '==============================================================================
91  Private Function ResolveComponent(ByVal workbook As Workbook, ByVal componentName As String) As Object
92     Dim vbProject As Object
93
94     On Error GoTo Missing
95        Set vbProject = workbook.VBProject
96        Set ResolveComponent = vbProject.VBComponents(componentName)
97     On Error GoTo 0
98     Exit Function
99
100 Missing:
101    ThrowError ProjectError.ElementNotFound, _
102            "Component '" & componentName & "' not found in workbook '" & workbook.Name & "'"
103 End Function
104
105 Private Function EnsureTargetComponent(ByVal workbook As Workbook, ByVal componentName As String) As Object
106    If ComponentExists(workbook, componentName) Then
107       Set EnsureTargetComponent = workbook.VBProject.VBComponents(componentName)
108    Else
109       ThrowError ProjectError.ElementNotFound, _
110                "Target workbook must contain component '" & componentName & "' when VBIDE automation is unavailable"
111    End If
```

```
End Function

Private Function ComponentExists(ByVal workbook As Workbook, ByVal componentName As String) As Boolean
   On Error GoTo Missing
      workbook.VBProject.VBComponents componentName
      ComponentExists = True
      Exit Function
Missing:
   Err.Clear
   ComponentExists = False
End Function

Private Sub ThrowError(ByVal errNumber As ProjectError, ByVal message As String)
   Err.Raise errNumber, CLASSNAME, message
End Sub


'@section Interface implementation
'==============================================================================
Private Sub ILinelistCodeTransferStrategy_TransferComponent(ByVal componentName As String, _
                                                            ByVal scope As LinelistCodeScope, _
                                                            ByVal source As Workbook, _
                                                            ByVal target As Workbook)
   TransferComponent componentName, scope, source, target
End Sub

Private Sub ILinelistCodeTransferStrategy_TransferWorkbookModule(ByVal moduleName As String, _
                                                                 ByVal source As Workbook, _
                                                                 ByVal target As Workbook)
   TransferWorkbookModule moduleName, source, target
End Sub

Private Sub ILinelistCodeTransferStrategy_TransferForm(ByVal formName As String, _
                                                       ByVal source As Workbook, _
                                                       ByVal target As Workbook)
   TransferForm formName, source, target
End Sub

Private Property Get ILinelistCodeTransferStrategy_Name() As String
   ILinelistCodeTransferStrategy_Name = Name
End Property
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/LinelistLifecycleManager.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1   'True
END
Attribute VB_Name = "LinelistLifecycleManager"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Attribute VB_Description = "Handles workbook disposal and reset operations for the Linelist workflow"

'@Folder("Linelist")
'@ModuleDescription("Handles workbook disposal and reset operations for the Linelist workflow")
'@IgnoreModule UnrecognizedAnnotation, UseMeaningfulName, ProcedureNotUsed
'@depends ProjectError

Option Explicit

Implements ILinelistLifecycleManager

'@section State and constants
'==============================================================================
Private Type TLifecycle
   accessor As ILinelistWorkbookAccessor
   releaser As ILinelistWorkbookReleaser
   tempFiles As ITemporaryRepos
   stateScope As IApplicationState
End Type

Private this As TLifecycle
Private Const CLASSNAME As String = "LinelistLifecycleManager"


'@section Factory helpers
'==============================================================================
Public Function Create(ByVal accessor As ILinelistWorkbookAccessor, _
                       ByVal tempFiles As ITemporaryRepos, _
                       Optional ByVal stateScope As IApplicationState, _
```

```
                Optional ByVal releaser As ILinelistWorkbookReleaser) As ILinelistLifecycleManager

    Dim manager As LinelistLifecycleManager
    Set manager = New LinelistLifecycleManager
    manager.Initialise accessor, tempFiles, stateScope, releaser
    Set Create = manager
End Function

Friend Sub Initialise(ByVal accessor As ILinelistWorkbookAccessor, _
                ByVal tempFiles As ITemporaryRepos, _
                ByVal stateScope As IApplicationState, _
                ByVal releaser As ILinelistWorkbookReleaser)

    EnsureAccessor accessor
    EnsureTempService tempFiles

    Set this.accessor = accessor
    Set this.tempFiles = tempFiles
    Set this.stateScope = stateScope

    If releaser Is Nothing Then
        Set this.releaser = TryCastReleaser(accessor)
    Else
        Set this.releaser = releaser
    End If
End Sub


'@section Lifecycle helpers
'==============================================================================
Private Sub ResetInternal(ByVal saveChanges As Boolean)
    Dim workbook As Workbook

    EnsureDependencies
    Set workbook = ResolveWorkbook

    CloseWorkbook workbook, saveChanges
    ClearWorkbookReference

    CleanupTemporaryFiles
    RefreshScope
End Sub

Private Sub CleanupTemporaryFiles()
    On Error GoTo CleanupFail
        this.tempFiles.DeleteAll
        this.tempFiles.Reset
        On Error GoTo 0
        Exit Sub

CleanupFail:
    ThrowError ProjectError.ErrorUnexpectedState, _
            "Failed to clean temporary files: " & Err.Description
End Sub

Private Sub RefreshScope()
    If this.stateScope Is Nothing Then Exit Sub
    this.stateScope.RefreshSnapshot
End Sub

Private Sub CloseWorkbook(ByVal workbook As Workbook, ByVal saveChanges As Boolean)
    If workbook Is Nothing Then Exit Sub

    On Error GoTo CloseFail
        workbook.Close SaveChanges:=saveChanges
        On Error GoTo 0
        Exit Sub

CloseFail:
    ThrowError ProjectError.ErrorUnexpectedState, _
            "Failed to close linelist workbook: " & Err.Description
End Sub

Private Sub ClearWorkbookReference()
    If Not this.releaser Is Nothing Then
        this.releaser.ClearOutputWorkbook
    End If
End Sub

Private Function ResolveWorkbook() As Workbook
    Dim workbook As Workbook

    Set workbook = this.accessor.OutputWorkbook
    Set ResolveWorkbook = workbook
End Function
```

```
'@section Validation helpers
'==============================================================================
Private Sub EnsureAccessor(ByVal accessor As ILinelistWorkbookAccessor)
    If accessor Is Nothing Then
        ThrowError ProjectError.ObjectNotInitialized, "Workbook accessor cannot be Nothing"
    End If
End Sub

Private Sub EnsureTempService(ByVal tempFiles As ITemporaryRepos)
    If tempFiles Is Nothing Then
        ThrowError ProjectError.ObjectNotInitialized, "Temporary file service cannot be Nothing"
    End If
End Sub

Private Sub EnsureDependencies()
    If this.accessor Is Nothing Then
        ThrowError ProjectError.ObjectNotInitialized, "Lifecycle manager was disposed"
    End If

    If this.tempFiles Is Nothing Then
        ThrowError ProjectError.ObjectNotInitialized, "Temporary file service is not available"
    End If
End Sub

Private Function TryCastReleaser(ByVal accessor As ILinelistWorkbookAccessor) As ILinelistWorkbookReleaser
    Dim releaser As ILinelistWorkbookReleaser

    On Error GoTo Missing
        Set releaser = accessor
        On Error GoTo 0
        Set TryCastReleaser = releaser
        Exit Function

Missing:
    Err.Clear
    Set TryCastReleaser = Nothing
End Function

Private Sub ThrowError(ByVal errNumber As ProjectError, ByVal message As String)
    Err.Raise errNumber, CLASSNAME, message
End Sub


'@section Interface implementation
'==============================================================================
Private Sub ILinelistLifecycleManager_Reset(Optional ByVal saveChanges As Boolean = False)
    ResetInternal saveChanges
End Sub

Private Sub ILinelistLifecycleManager_Dispose(Optional ByVal saveChanges As Boolean = False)
    ResetInternal saveChanges

    Set this.accessor = Nothing
    Set this.releaser = Nothing
    Set this.tempFiles = Nothing
    Set this.stateScope = Nothing
End Sub
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/LinelistPreparationContext.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1   'True
END
Attribute VB_Name = "LinelistPreparationContext"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Attribute VB_Description = "Holds shared state while the Linelist preparation pipeline executes"

'@Folder("Linelist")
'@ModuleDescription("Holds shared state while the Linelist preparation pipeline executes")
'@IgnoreModule UnrecognizedAnnotation, UseMeaningfulName, HungarianNotation
'@depends ProjectError

Option Explicit

Implements ILinelistPreparationContext

'@section State and constants
'==============================================================================
```

```
23  Private Type TContext
24     linelist As ILinelist
25     specs As ILinelistSpecs
26     dictionary As ILLdictionary
27     sheetFormatter As ILinelistSheetNameFormatter
28     stateScope As IApplicationState
29  End Type
30
31  Private this As TContext
32  Private Const CLASSNAME As String = "LinelistPreparationContext"
33
34
35  '@section Factory helpers
36  '===============================================================================
37  Public Function Create(ByVal linelist As ILinelist, _
38                         ByVal specs As ILinelistSpecs, _
39                         ByVal dictionary As ILLdictionary, _
40                         ByVal formatter As ILinelistSheetNameFormatter, _
41                         ByVal scope As IApplicationState) As ILinelistPreparationContext
42
43     Dim context As LinelistPreparationContext
44     Set context = New LinelistPreparationContext
45     context.Initialise linelist, specs, dictionary, formatter, scope
46     Set Create = context
47  End Function
48
49  Friend Sub Initialise(ByVal linelist As ILinelist, _
50                        ByVal specs As ILinelistSpecs, _
51                        ByVal dictionary As ILLdictionary, _
52                        ByVal formatter As ILinelistSheetNameFormatter, _
53                        ByVal scope As IApplicationState)
54
55     ValidateLinelist linelist
56     ValidateSpecs specs
57     ValidateDictionary dictionary
58     ValidateFormatter formatter
59     ValidateScope scope
60
61     Set this.linelist = linelist
62     Set this.specs = specs
63     Set this.dictionary = dictionary
64     Set this.sheetFormatter = formatter
65     Set this.stateScope = scope
66  End Sub
67
68
69  '@section Validation helpers
70  '===============================================================================
71  Private Sub ValidateLinelist(ByVal linelist As ILinelist)
72     If linelist Is Nothing Then
73        ThrowError ProjectError.ObjectNotInitialized, "Linelist reference cannot be Nothing"
74     End If
75  End Sub
76
77  Private Sub ValidateSpecs(ByVal specs As ILinelistSpecs)
78     If specs Is Nothing Then
79        ThrowError ProjectError.ObjectNotInitialized, "Specifications reference cannot be Nothing"
80     End If
81  End Sub
82
83  Private Sub ValidateDictionary(ByVal dictionary As ILLdictionary)
84     If dictionary Is Nothing Then
85        ThrowError ProjectError.ObjectNotInitialized, "Dictionary reference cannot be Nothing"
86     End If
87  End Sub
88
89  Private Sub ValidateFormatter(ByVal formatter As ILinelistSheetNameFormatter)
90     If formatter Is Nothing Then
91        ThrowError ProjectError.ObjectNotInitialized, "Sheet formatter reference cannot be Nothing"
92     End If
93  End Sub
94
95  Private Sub ValidateScope(ByVal scope As IApplicationState)
96     If scope Is Nothing Then
97        ThrowError ProjectError.ObjectNotInitialized, "Application state scope cannot be Nothing"
98     End If
99  End Sub
100
101 Private Sub ThrowError(ByVal errNumber As ProjectError, ByVal message As String)
102    Err.Raise errNumber, CLASSNAME, message
103 End Sub
104
105
106 '@section Interface implementation
107 '===============================================================================
108 Private Property Get ILinelistPreparationContext_Linelist() As ILinelist
109    Set ILinelistPreparationContext_Linelist = this.linelist
```

```
End Property

Private Property Get ILinelistPreparationContext_Specifications() As ILinelistSpecs
    Set ILinelistPreparationContext_Specifications = this.specs
End Property

Private Property Get ILinelistPreparationContext_Dictionary() As ILLdictionary
    Set ILinelistPreparationContext_Dictionary = this.dictionary
End Property

Private Property Get ILinelistPreparationContext_SheetFormatter() As ILinelistSheetNameFormatter
    Set ILinelistPreparationContext_SheetFormatter = this.sheetFormatter
End Property

Private Property Get ILinelistPreparationContext_StateScope() As IApplicationState
    Set ILinelistPreparationContext_StateScope = this.stateScope
End Property
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/LinelistPreparationPipeline.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1   'True
END
Attribute VB_Name = "LinelistPreparationPipeline"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Attribute VB_Description = "Coordinates Linelist preparation through explicit, testable stages"

'@Folder("Linelist")
'@ModuleDescription("Coordinates Linelist preparation through explicit, testable stages")
'@IgnoreModule UnrecognizedAnnotation, UseMeaningfulName, HungarianNotation
'@depends ProjectError

Option Explicit

Implements ILinelistPreparationPipeline

'@section Stage enumeration
'==============================================================================
Public Enum LinelistPreparationStage
    stageCreateTemporarySheets = 1
    stageExportSpecifications = 2
    stageCreateAnalysisSheets = 3
    stageTransferCodeModules = 4
    stageTransferForms = 5
End Enum

'@section State and constants
'==============================================================================
Private Type TPipeline
    context As ILinelistPreparationContext
    steps As Collection                    'Registered steps keyed by stage
    orderedStages As BetterArray                'Execution order
End Type

Private this As TPipeline
Private Const CLASSNAME As String = "LinelistPreparationPipeline"


'@section Factory helpers
'==============================================================================
Public Function Create(ByVal linelist As ILinelist, _
                       ByVal specs As ILinelistSpecs, _
                       ByVal temporarySheets As ILinelistPreparationStep, _
                       ByVal specExporter As ILinelistPreparationStep, _
                       ByVal analysisSheets As ILinelistPreparationStep, _
                       ByVal codeTransfer As ILinelistPreparationStep, _
                       ByVal formTransfer As ILinelistPreparationStep, _
                       Optional ByVal formatter As ILinelistSheetNameFormatter, _
                       Optional ByVal stateScope As IApplicationState) As ILinelistPreparationPipeline

    Dim pipeline As LinelistPreparationPipeline
    Set pipeline = New LinelistPreparationPipeline
    pipeline.Initialise linelist, specs, temporarySheets, specExporter, analysisSheets, codeTransfer, formTransfer, formatter, stateScope
    Set Create = pipeline
End Function

Friend Sub Initialise(ByVal linelist As ILinelist, _
                      ByVal specs As ILinelistSpecs, _
                      ByVal temporarySheets As ILinelistPreparationStep, _
```

```
64                  ByVal specExporter As ILinelistPreparationStep, _
65                  ByVal analysisSheets As ILinelistPreparationStep, _
66                  ByVal codeTransfer As ILinelistPreparationStep, _
67                  ByVal formTransfer As ILinelistPreparationStep, _
68                  ByVal formatter As ILinelistSheetNameFormatter, _
69                  ByVal stateScope As IApplicationState)
70
71      Dim dictionary As ILLdictionary
72      Dim actualFormatter As ILinelistSheetNameFormatter
73      Dim actualScope As IApplicationState
74
75      EnsureLinelist linelist
76      Set dictionary = ResolveDictionary(specs)
77      Set actualFormatter = ResolveFormatter(formatter)
78      Set actualScope = ResolveScope(stateScope)
79
80      Set this.context = LinelistPreparationContext.Create(linelist, specs, dictionary, actualFormatter, actualScope)
81      Set this.steps = New Collection
82      Set this.orderedStages = New BetterArray
83
84      RegisterStage stageCreateTemporarySheets, temporarySheets
85      RegisterStage stageExportSpecifications, specExporter
86      RegisterStage stageCreateAnalysisSheets, analysisSheets
87      RegisterStage stageTransferCodeModules, codeTransfer
88      RegisterStage stageTransferForms, formTransfer
89  End Sub
90
91
92  '@section Validation helpers
93  '==============================================================================
94  Private Sub EnsureLinelist(ByVal linelist As ILinelist)
95      If linelist Is Nothing Then
96          ThrowError ProjectError.ObjectNotInitialized, "Linelist reference cannot be Nothing"
97      End If
98  End Sub
99
100 Private Function ResolveDictionary(ByVal specs As ILinelistSpecs) As ILLdictionary
101     Dim dictionary As ILLdictionary
102
103     If specs Is Nothing Then
104         ThrowError ProjectError.ObjectNotInitialized, "Specifications reference cannot be Nothing"
105     End If
106
107     Set dictionary = specs.Dictionary
108     If dictionary Is Nothing Then
109         ThrowError ProjectError.ObjectNotInitialized, "Specifications must expose a dictionary"
110     End If
111
112     Set ResolveDictionary = dictionary
113 End Function
114
115 Private Function ResolveFormatter(ByVal formatter As ILinelistSheetNameFormatter) As ILinelistSheetNameFormatter
116     If formatter Is Nothing Then
117         Set ResolveFormatter = LinelistSheetNameFormatter.Create
118     Else
119         Set ResolveFormatter = formatter
120     End If
121 End Function
122
123 Private Function ResolveScope(ByVal scope As IApplicationState) As IApplicationState
124     If scope Is Nothing Then
125         Set ResolveScope = ApplicationState.Create(Application)
126     Else
127         Set ResolveScope = scope
128     End If
129 End Function
130
131 Private Sub RegisterStage(ByVal stage As LinelistPreparationStage, ByVal step As ILinelistPreparationStep)
132     If step Is Nothing Then
133         ThrowError ProjectError.ObjectNotInitialized, "Preparation step cannot be Nothing"
134     End If
135
136     If HasStage(stage) Then
137         ThrowError ProjectError.ElementShouldNotExists, "Stage already registered"
138     End If
139
140     this.steps.Add step, CStr(stage)
141     this.orderedStages.Push stage
142 End Sub
143
144 Private Sub ThrowError(ByVal errNumber As ProjectError, ByVal message As String)
145     Err.Raise errNumber, CLASSNAME, message
146 End Sub
147
148
149 '@section Stage execution helpers
150 '==============================================================================
```

```
151 Private Sub CreateTemporarySheets()
152     ExecuteStage stageCreateTemporarySheets
153 End Sub
154
155 Private Sub ExportSpecifications()
156     ExecuteStage stageExportSpecifications
157 End Sub
158
159 Private Sub CreateAnalysisSheets()
160     ExecuteStage stageCreateAnalysisSheets
161 End Sub
162
163 Private Sub TransferCodeModules()
164     ExecuteStage stageTransferCodeModules
165 End Sub
166
167 Private Sub TransferForms()
168     ExecuteStage stageTransferForms
169 End Sub
170
171 Private Sub ExecuteStage(ByVal stage As LinelistPreparationStage)
172     Dim stepObject As ILinelistPreparationStep
173
174     Set stepObject = FetchStage(stage)
175     stepObject.Execute this.context
176 End Sub
177
178
179 '@section Interface implementation
180 '==============================================================================
181 Private Sub ILinelistPreparationPipeline_Prepare()
182     Dim scope As IApplicationState
183     Dim stageIndex As Long
184     Dim stageId As LinelistPreparationStage
185     Dim stageName As String
186
187     Set scope = this.context.StateScope
188     scope.ApplyBusyState
189
190     On Error GoTo PrepareFail
191         For stageIndex = this.orderedStages.LowerBound To this.orderedStages.UpperBound
192             stageId = CLng(this.orderedStages.Item(stageIndex))
193             stageName = StageName(stageId)
194             Select Case stageId
195                 Case stageCreateTemporarySheets: CreateTemporarySheets
196                 Case stageExportSpecifications: ExportSpecifications
197                 Case stageCreateAnalysisSheets: CreateAnalysisSheets
198                 Case stageTransferCodeModules: TransferCodeModules
199                 Case stageTransferForms: TransferForms
200                 Case Else
201                     ThrowError ProjectError.InvalidArgument, _
202                         "Unknown preparation stage encountered during execution"
203             End Select
204         Next stageIndex
205     On Error GoTo 0
206
207     scope.Restore
208     Exit Sub
209
210 PrepareFail:
211     scope.Restore
212     ThrowError ProjectError.ErrorUnexpectedState, _
213             "Preparation failed during stage " & stageName & ": " & Err.Description
214 End Sub
215
216 Private Function HasStage(ByVal stage As LinelistPreparationStage) As Boolean
217     On Error GoTo Missing
218         this.steps.Item CStr(stage)
219         HasStage = True
220         Exit Function
221 Missing:
222     Err.Clear
223     HasStage = False
224 End Function
225
226 Private Function FetchStage(ByVal stage As LinelistPreparationStage) As ILinelistPreparationStep
227     On Error GoTo Missing
228         Set FetchStage = this.steps.Item(CStr(stage))
229         Exit Function
230 Missing:
231     ThrowError ProjectError.ElementNotFound, "Preparation stage not registered"
232 End Function
233
234 Private Function StageName(ByVal stage As LinelistPreparationStage) As String
235     Select Case stage
236         Case stageCreateTemporarySheets: StageName = "CreateTemporarySheets"
237         Case stageExportSpecifications: StageName = "ExportSpecifications"
```

```
238         Case stageCreateAnalysisSheets: StageName = "CreateAnalysisSheets"
239         Case stageTransferCodeModules: StageName = "TransferCodeModules"
240         Case stageTransferForms: StageName = "TransferForms"
241         Case Else: StageName = CStr(stage)
242     End Select
243 End Function
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/LinelistSaveWorkflow.cls

```
1   VERSION 1.0 CLASS
2   BEGIN
3     MultiUse = -1  'True
4   END
5   Attribute VB_Name = "LinelistSaveWorkflow"
6   Attribute VB_GlobalNameSpace = False
7   Attribute VB_Creatable = False
8   Attribute VB_PredeclaredId = False
9   Attribute VB_Exposed = False
10  Attribute VB_Description = "Handles final save sequence for the Linelist workbook"
11
12  '@Folder("Linelist")
13  '@ModuleDescription("Handles final save sequence for the Linelist workbook")
14  '@IgnoreModule UnrecognizedAnnotation, UseMeaningfulName, ProcedureNotUsed
15  '@depends ProjectError
16
17  Option Explicit
18
19  Implements ILinelistSaveWorkflow
20
21  '@section State and constants
22  '==============================================================================
23  Private Type TSaveWorkflow
24      navigator As ILinelistSheetNavigator
25      lifecycle As ILinelistLifecycleManager
26  End Type
27
28  Private this As TSaveWorkflow
29  Private Const CLASSNAME As String = "LinelistSaveWorkflow"
30
31
32  '@section Factory helpers
33  '==============================================================================
34  Public Function Create(ByVal navigator As ILinelistSheetNavigator, _
35                         ByVal lifecycle As ILinelistLifecycleManager) As ILinelistSaveWorkflow
36
37      Dim workflow As LinelistSaveWorkflow
38      Set workflow = New LinelistSaveWorkflow
39      workflow.Initialise navigator, lifecycle
40      Set Create = workflow
41  End Function
42
43  Friend Sub Initialise(ByVal navigator As ILinelistSheetNavigator, _
44                        ByVal lifecycle As ILinelistLifecycleManager)
45
46      EnsureNavigator navigator
47      EnsureLifecycle lifecycle
48
49      Set this.navigator = navigator
50      Set this.lifecycle = lifecycle
51  End Sub
52
53
54  '@section Core workflow
55  '==============================================================================
56  Private Sub ExecuteSave(ByVal context As ILinelistPreparationContext)
57      Dim specs As ILinelistSpecs
58      Dim mainObj As IMain
59      Dim passwordProvider As IPasswords
60      Dim workbook As Workbook
61      Dim outputPath As String
62      Dim password As String
63      Dim saveResult As Boolean
64
65      Set specs = EnsureSpecifications(context)
66      Set mainObj = EnsureMainObject(specs)
67      Set passwordProvider = EnsurePasswordProvider(specs)
68      Set workbook = EnsureWorkbook(context)
69
70      outputPath = BuildOutputPath(mainObj)
71      password = mainObj.ValueOf("llpassword")
72
73      ActivateSheets
```

```
74      passwordProvider.ProtectWkb workbook
75      saveResult = SaveWorkbook(workbook, outputPath, password)
76
77      If Not saveResult Then Exit Sub
78
79      this.lifecycle.Dispose False
80  End Sub
81
82  Private Sub ActivateSheets()
83      On Error GoTo ActivationFail
84          this.navigator.ActivateAdminSheet
85          this.navigator.ActivateInstructionSheet
86          On Error GoTo 0
87          Exit Sub
88
89  ActivationFail:
90      ThrowError ProjectError.ErrorUnexpectedState, _
91              "Failed to activate sheets prior to saving: " & Err.Description
92  End Sub
93
94  Private Function SaveWorkbook(ByVal workbook As Workbook, _
95                                ByVal path As String, _
96                                ByVal password As String) As Boolean
97      On Error GoTo SaveFail
98          workbook.SaveAs fileName:=path, _
99                          FileFormat:=xlExcel12, _
100                         Password:=password, _
101                         ConflictResolution:=Excel.XlSaveConflictResolution.xlLocalSessionChanges
102         SaveWorkbook = True
103         Exit Function
104
105 SaveFail:
106     ThrowError ProjectError.ErrorUnexpectedState, _
107             "Failed to save linelist workbook: " & Err.Description
108 End Function
109
110 Private Function BuildOutputPath(ByVal mainObj As IMain) As String
111     Dim directoryPath As String
112     Dim fileName As String
113
114     directoryPath = mainObj.ValueOf("lldir")
115     fileName = mainObj.ValueOf("llname")
116
117     If LenB(Trim$(directoryPath)) = 0 Or LenB(Trim$(fileName)) = 0 Then
118         ThrowError ProjectError.InvalidArgument, "Linelist directory/name must be defined before saving"
119     End If
120
121     BuildOutputPath = directoryPath & Application.PathSeparator & fileName & ".xlsb"
122 End Function
123
124
125 '@section Dependency validation
126 '==============================================================================
127 Private Function EnsureSpecifications(ByVal context As ILinelistPreparationContext) As ILinelistSpecs
128     Dim specs As ILinelistSpecs
129
130     If context Is Nothing Then
131         ThrowError ProjectError.ObjectNotInitialized, "Save context cannot be Nothing"
132     End If
133
134     Set specs = context.Specifications
135     If specs Is Nothing Then
136         ThrowError ProjectError.ObjectNotInitialized, "Save context must expose specifications"
137     End If
138
139     Set EnsureSpecifications = specs
140 End Function
141
142 Private Function EnsureMainObject(ByVal specs As ILinelistSpecs) As IMain
143     Dim mainObj As IMain
144
145     Set mainObj = specs.MainObject
146     If mainObj Is Nothing Then
147         ThrowError ProjectError.ObjectNotInitialized, "Specifications must provide a main object"
148     End If
149
150     Set EnsureMainObject = mainObj
151 End Function
152
153 Private Function EnsurePasswordProvider(ByVal specs As ILinelistSpecs) As IPasswords
154     Dim provider As IPasswords
155
156     Set provider = specs.Password
157     If provider Is Nothing Then
158         ThrowError ProjectError.ObjectNotInitialized, "Specifications must provide a password provider"
159     End If
160
```

```
    Set EnsurePasswordProvider = provider
End Function

Private Function EnsureWorkbook(ByVal context As ILinelistPreparationContext) As Workbook
    Dim linelist As ILinelist
    Dim accessor As ILinelistWorkbookAccessor
    Dim workbook As Workbook

    Set linelist = context.Linelist
    If linelist Is Nothing Then
        ThrowError ProjectError.ObjectNotInitialized, "Context must expose a linelist reference"
    End If

    On Error GoTo MissingAccessor
        Set accessor = linelist
        On Error GoTo 0
        Set workbook = accessor.OutputWorkbook
        If workbook Is Nothing Then
            ThrowError ProjectError.ObjectNotInitialized, "Linelist output workbook cannot be Nothing"
        End If
        Set EnsureWorkbook = workbook
        Exit Function

MissingAccessor:
    Err.Clear
    ThrowError ProjectError.InvalidOperation, "Linelist does not expose ILinelistWorkbookAccessor"
End Function

Private Sub EnsureNavigator(ByVal navigator As ILinelistSheetNavigator)
    If navigator Is Nothing Then
        ThrowError ProjectError.ObjectNotInitialized, "Sheet navigator cannot be Nothing"
    End If
End Sub

Private Sub EnsureLifecycle(ByVal lifecycle As ILinelistLifecycleManager)
    If lifecycle Is Nothing Then
        ThrowError ProjectError.ObjectNotInitialized, "Lifecycle manager cannot be Nothing"
    End If
End Sub

Private Sub ThrowError(ByVal errNumber As ProjectError, ByVal message As String)
    Err.Raise errNumber, CLASSNAME, message
End Sub


'@section Interface implementation
'==============================================================================
Private Sub ILinelistSaveWorkflow_Save(ByVal context As ILinelistPreparationContext)
    ExecuteSave context
End Sub
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/LinelistSheetNameFormatter.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1   'True
END
Attribute VB_Name = "LinelistSheetNameFormatter"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = True
Attribute VB_Exposed = False
Attribute VB_Description = "Formats Linelist sheet names consistently across scopes"

'@PredeclaredId
'@Folder("Linelist")
'@ModuleDescription("Formats Linelist sheet names consistently across scopes")
'@IgnoreModule UnrecognizedAnnotation, UseMeaningfulName, ProcedureNotUsed, HungarianNotation
'@depends ProjectError

Option Explicit

Implements ILinelistSheetNameFormatter

'@section State and constants
'==============================================================================
Private Type TFormatter
    sheetNameMaxLength As Long              'Maximum supported length for sheet names including scope prefix
End Type

Private this As TFormatter
```

```
29  Private Const CLASSNAME As String = "LinelistSheetNameFormatter"
30  Private Const PREFIX_PRINT As String = "print_"
31  Private Const PREFIX_CRF As String = "crf_"
32  Private Const DEFAULT_SHEET_MAX_LENGTH As Long = 30
33
34
35  '@section Factory helpers
36  '==============================================================================
37  '@sub-title Create a formatter instance with default configuration
38  Public Function Create() As ILinelistSheetNameFormatter
39      Dim formatter As LinelistSheetNameFormatter
40
41      Set formatter = New LinelistSheetNameFormatter
42      formatter.Initialise
43
44      Set Create = formatter.Self
45  End Function
46
47  '@sub-title Self reference
48  Public Property Get Self() As ILinelistSheetNameFormatter
49      Set Self = Me
50  End Property
51
52  '@sub-title Initialise internal state
53  Friend Sub Initialise()
54      this.sheetNameMaxLength = DEFAULT_SHEET_MAX_LENGTH
55  End Sub
56
57
58  '@section Sheet logic
59  '==============================================================================
60  '@sub-title Build the scoped sheet name ensuring the length budget is honoured
61  Public Function FormatSheetName(ByVal baseName As String, _
62                                  Optional ByVal scope As LinelistSheetScope = sheetScopeStandard) As String
63      FormatSheetName = FormatSheetNameCore(baseName, scope)
64  End Function
65
66  '@sub-title Determine whether a scoped sheet exists inside a workbook
67  Public Function SheetExists(ByVal workbook As Workbook, ByVal baseName As String, _
68                              Optional ByVal scope As LinelistSheetScope = sheetScopeStandard) As Boolean
69      SheetExists = SheetExistsCore(workbook, baseName, scope)
70  End Function
71
72  '@sub-title Retrieve a scoped worksheet reference from the supplied workbook
73  Public Function ResolveWorksheet(ByVal workbook As Workbook, ByVal baseName As String, _
74                                   Optional ByVal scope As LinelistSheetScope = sheetScopeStandard) As Worksheet
75      Set ResolveWorksheet = ResolveWorksheetCore(workbook, baseName, scope)
76  End Function
77
78
79  '@section Accessors
80  '==============================================================================
81  Public Property Get PrintPrefix() As String
82      PrintPrefix = PREFIX_PRINT
83  End Property
84
85  Public Property Get CrfPrefix() As String
86      CrfPrefix = PREFIX_CRF
87  End Property
88
89  Public Property Get SheetNameMaxLength() As Long
90      SheetNameMaxLength = this.sheetNameMaxLength
91  End Property
92
93
94  '@section Core helpers
95  '==============================================================================
96  Private Function FormatSheetNameCore(ByVal baseName As String, _
97                                       ByVal scope As LinelistSheetScope) As String
98      Dim trimmedName As String
99      Dim prefix As String
100     Dim candidate As String
101
102     trimmedName = Trim$(baseName)
103     If LenB(trimmedName) = 0 Then
104         ThrowError ProjectError.InvalidArgument, "Sheet base name cannot be empty"
105     End If
106
107     prefix = ScopePrefix(scope)
108     candidate = prefix & trimmedName
109
110     If Len(candidate) > this.sheetNameMaxLength Then
111         candidate = Left$(candidate, this.sheetNameMaxLength)
112     End If
113
114     FormatSheetNameCore = candidate
115 End Function
```

```
116
117 Private Function SheetExistsCore(ByVal workbook As Workbook, _
118                                ByVal baseName As String, _
119                                ByVal scope As LinelistSheetScope) As Boolean
120     Dim sheetName As String
121     Dim targetWorkbook As Workbook
122     Dim targetSheet As Worksheet
123
124     Set targetWorkbook = EnsureWorkbook(workbook)
125     sheetName = FormatSheetNameCore(baseName, scope)
126
127     On Error Resume Next
128         Set targetSheet = targetWorkbook.Worksheets(sheetName)
129     On Error GoTo 0
130
131     SheetExistsCore = Not targetSheet Is Nothing
132 End Function
133
134 Private Function ResolveWorksheetCore(ByVal workbook As Workbook, _
135                                      ByVal baseName As String, _
136                                      ByVal scope As LinelistSheetScope) As Worksheet
137     Dim sheetName As String
138     Dim targetWorkbook As Workbook
139
140     Set targetWorkbook = EnsureWorkbook(workbook)
141     sheetName = FormatSheetNameCore(baseName, scope)
142
143     On Error Resume Next
144         Set ResolveWorksheetCore = targetWorkbook.Worksheets(sheetName)
145     On Error GoTo 0
146
147     If ResolveWorksheetCore Is Nothing Then
148         ThrowError ProjectError.ElementNotFound, _
149                 "Worksheet '" & sheetName & "' not found in workbook '" & targetWorkbook.Name & "'"
150     End If
151 End Function
152
153 Private Function EnsureWorkbook(ByVal workbook As Workbook) As Workbook
154     If workbook Is Nothing Then
155         ThrowError ProjectError.ObjectNotInitialized, "Workbook reference cannot be Nothing"
156     End If
157
158     If StrComp(TypeName(workbook), "Workbook", vbTextCompare) <> 0 Then
159         ThrowError ProjectError.InvalidArgument, "Workbook argument must be an Excel Workbook"
160     End If
161
162     Set EnsureWorkbook = workbook
163 End Function
164
165 Private Function ScopePrefix(ByVal scope As LinelistSheetScope) As String
166     Select Case scope
167         Case sheetScopeStandard
168             ScopePrefix = vbNullString
169         Case sheetScopePrint
170             ScopePrefix = PREFIX_PRINT
171         Case sheetScopeCrf
172             ScopePrefix = PREFIX_CRF
173         Case Else
174             ThrowError ProjectError.InvalidArgument, _
175                     "Unknown sheet scope value: " & CStr(scope)
176     End Select
177 End Function
178
179 Private Sub ThrowError(ByVal errNumber As ProjectError, ByVal message As String)
180     Err.Raise errNumber, CLASSNAME, message
181 End Sub
182
183
184 '@section Interface implementation
185 '=============================================================================
186 Private Function ILinelistSheetNameFormatter_FormatSheetName(ByVal baseName As String, _
187                                                             Optional ByVal scope As LinelistSheetScope = sheetScopeStandard) As String
188     ILinelistSheetNameFormatter_FormatSheetName = FormatSheetNameCore(baseName, scope)
189 End Function
190
191 Private Function ILinelistSheetNameFormatter_SheetExists(ByVal workbook As Workbook, ByVal baseName As String, _
192                                                         Optional ByVal scope As LinelistSheetScope = sheetScopeStandard) As Boolean
193     ILinelistSheetNameFormatter_SheetExists = SheetExistsCore(workbook, baseName, scope)
194 End Function
195
196 Private Function ILinelistSheetNameFormatter_ResolveWorksheet(ByVal workbook As Workbook, ByVal baseName As String, _
197                                                              Optional ByVal scope As LinelistSheetScope = sheetScopeStandard) As Worksheet
198     Set ILinelistSheetNameFormatter_ResolveWorksheet = ResolveWorksheetCore(workbook, baseName, scope)
199 End Function
200
201 Private Property Get ILinelistSheetNameFormatter_PrintPrefix() As String
202     ILinelistSheetNameFormatter_PrintPrefix = PREFIX_PRINT
```

```
End Property

Private Property Get ILinelistSheetNameFormatter_CrfPrefix() As String
    ILinelistSheetNameFormatter_CrfPrefix = PREFIX_CRF
End Property

Private Property Get ILinelistSheetNameFormatter_SheetNameMaxLength() As Long
    ILinelistSheetNameFormatter_SheetNameMaxLength = this.sheetNameMaxLength
End Property
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/LinelistSheetNavigator.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1   'True
END
Attribute VB_Name = "LinelistSheetNavigator"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Attribute VB_Description = "Safely activates linelist worksheets using cached translations and formatted names"

'@Folder("Linelist")
'@ModuleDescription("Safely activates linelist worksheets using cached translations and formatted names")
'@IgnoreModule UnrecognizedAnnotation, UseMeaningfulName, ProcedureNotUsed
'@depends ProjectError

Option Explicit

Implements ILinelistSheetNavigator

'@section State and constants
'==============================================================================
Private Type TNavigator
    workbook As Workbook
    formatter As ILinelistSheetNameFormatter
    translations As ITranslationObject
    adminBaseName As String
    instructionBaseName As String
    cacheInitialised As Boolean
    lastActivatedName As String
    translationCacheHits As Long
End Type

Private this As TNavigator
Private Const CLASSNAME As String = "LinelistSheetNavigator"
Private Const ADMIN_KEY As String = "LLSHEET_Admin"
Private Const INSTRUCTION_KEY As String = "INSTSHEETNAME"


'@section Factory helpers
'==============================================================================
Public Function Create(ByVal workbook As Workbook, _
                       ByVal translations As ITranslationObject, _
                       Optional ByVal formatter As ILinelistSheetNameFormatter) As ILinelistSheetNavigator

    Dim navigator As LinelistSheetNavigator

    Set navigator = New LinelistSheetNavigator
    navigator.Initialise workbook, translations, formatter

    Set Create = navigator
End Function

Friend Sub Initialise(ByVal workbook As Workbook, _
                      ByVal translations As ITranslationObject, _
                      ByVal formatter As ILinelistSheetNameFormatter)

    Set this.workbook = EnsureWorkbook(workbook)
    Set this.translations = EnsureTranslations(translations)
    Set this.formatter = ResolveFormatter(formatter)

    this.cacheInitialised = False
    this.lastActivatedName = vbNullString
    this.translationCacheHits = 0
End Sub


'@section Public helpers
'==============================================================================
Public Sub ActivateAdminSheet()
```

```
71      EnsureCoreNames
72      ActivateAndTrack this.adminBaseName, sheetScopeStandard
73  End Sub
74
75  Public Sub ActivateInstructionSheet()
76      EnsureCoreNames
77      ActivateAndTrack this.instructionBaseName, sheetScopeStandard
78  End Sub
79
80  Public Function ActivateSheet(ByVal baseName As String, _
81                                Optional ByVal scope As LinelistSheetScope = sheetScopeStandard) As Worksheet
82      Set ActivateSheet = ActivateAndTrack(baseName, scope)
83  End Function
84
85  Public Function ResolveSheet(ByVal baseName As String, _
86                               Optional ByVal scope As LinelistSheetScope = sheetScopeStandard) As Worksheet
87      Set ResolveSheet = ResolveSheetCore(baseName, scope)
88  End Function
89
90  Public Function SheetExists(ByVal baseName As String, _
91                              Optional ByVal scope As LinelistSheetScope = sheetScopeStandard) As Boolean
92      Dim trimmedName As String
93
94      trimmedName = NormaliseName(baseName)
95      If LenB(trimmedName) = 0 Then
96          ThrowError ProjectError.InvalidArgument, "Sheet base name cannot be empty"
97      End If
98
99      SheetExists = this.formatter.SheetExists(this.workbook, trimmedName, scope)
100 End Function
101
102 Public Property Get LastActivatedSheetName() As String
103     LastActivatedSheetName = this.lastActivatedName
104 End Property
105
106 Public Property Get TranslationCacheHits() As Long
107     TranslationCacheHits = this.translationCacheHits
108 End Property
109
110
111 '@section Core helpers
112 '==============================================================================
113 Private Function ActivateAndTrack(ByVal baseName As String, _
114                                  ByVal scope As LinelistSheetScope) As Worksheet
115     Dim sheet As Worksheet
116
117     Set sheet = ResolveSheetCore(baseName, scope)
118
119     On Error GoTo ActivateFail
120         sheet.Activate
121         this.lastActivatedName = sheet.Name
122         Set ActivateAndTrack = sheet
123         On Error GoTo 0
124         Exit Function
125
126 ActivateFail:
127     ThrowError ProjectError.ErrorUnexpectedState, _
128             "Unable to activate worksheet '" & sheet.Name & "': " & Err.Description
129 End Function
130
131 Private Function ResolveSheetCore(ByVal baseName As String, _
132                                   ByVal scope As LinelistSheetScope) As Worksheet
133     Dim trimmedName As String
134
135     trimmedName = NormaliseName(baseName)
136     If LenB(trimmedName) = 0 Then
137         ThrowError ProjectError.InvalidArgument, "Sheet base name cannot be empty"
138     End If
139
140     Set ResolveSheetCore = this.formatter.ResolveWorksheet(this.workbook, trimmedName, scope)
141 End Function
142
143 Private Sub EnsureCoreNames()
144     If this.cacheInitialised Then Exit Sub
145
146     this.adminBaseName = LookupTranslation(ADMIN_KEY)
147     this.instructionBaseName = LookupTranslation(INSTRUCTION_KEY)
148
149     this.cacheInitialised = True
150 End Sub
151
152 Private Function LookupTranslation(ByVal tagName As String) As String
153     Dim translated As String
154
155     translated = Trim$(CStr(this.translations.TranslatedValue(tagName)))
156
157     this.translationCacheHits = this.translationCacheHits + 1
```

```
158
159     If LenB(translated) = 0 Then
160         ThrowError ProjectError.ElementNotFound, _
161                 "Translation key '" & tagName & "' returned an empty value"
162     End If
163
164     LookupTranslation = translated
165 End Function
166
167 Private Function NormaliseName(ByVal baseName As String) As String
168     NormaliseName = Trim$(baseName)
169 End Function
170
171 Private Function EnsureWorkbook(ByVal workbook As Workbook) As Workbook
172     If workbook Is Nothing Then
173         ThrowError ProjectError.ObjectNotInitialized, "Workbook reference cannot be Nothing"
174     End If
175
176     If StrComp(TypeName(workbook), "Workbook", vbTextCompare) <> 0 Then
177         ThrowError ProjectError.InvalidArgument, "Navigator expects an Excel workbook"
178     End If
179
180     Set EnsureWorkbook = workbook
181 End Function
182
183 Private Function EnsureTranslations(ByVal translations As ITranslationObject) As ITranslationObject
184     If translations Is Nothing Then
185         ThrowError ProjectError.ObjectNotInitialized, "Translation object cannot be Nothing"
186     End If
187
188     Set EnsureTranslations = translations
189 End Function
190
191 Private Function ResolveFormatter(ByVal formatter As ILinelistSheetNameFormatter) As ILinelistSheetNameFormatter
192     If formatter Is Nothing Then
193         Set ResolveFormatter = LinelistSheetNameFormatter.Create
194     Else
195         Set ResolveFormatter = formatter
196     End If
197 End Function
198
199 Private Sub ThrowError(ByVal errNumber As ProjectError, ByVal message As String)
200     Err.Raise errNumber, CLASSNAME, message
201 End Sub
202
203
204 '@section Interface implementation
205 '==============================================================================
206 Private Sub ILinelistSheetNavigator_ActivateAdminSheet()
207     ActivateAdminSheet
208 End Sub
209
210 Private Sub ILinelistSheetNavigator_ActivateInstructionSheet()
211     ActivateInstructionSheet
212 End Sub
213
214 Private Function ILinelistSheetNavigator_ActivateSheet(ByVal baseName As String, _
215                                                       Optional ByVal scope As LinelistSheetScope = sheetScopeStandard) As Worksheet
216     Set ILinelistSheetNavigator_ActivateSheet = ActivateSheet(baseName, scope)
217 End Function
218
219 Private Function ILinelistSheetNavigator_ResolveSheet(ByVal baseName As String, _
220                                                      Optional ByVal scope As LinelistSheetScope = sheetScopeStandard) As Worksheet
221     Set ILinelistSheetNavigator_ResolveSheet = ResolveSheet(baseName, scope)
222 End Function
223
224 Private Function ILinelistSheetNavigator_SheetExists(ByVal baseName As String, _
225                                                     Optional ByVal scope As LinelistSheetScope = sheetScopeStandard) As Boolean
226     ILinelistSheetNavigator_SheetExists = SheetExists(baseName, scope)
227 End Function
228
229 Private Property Get ILinelistSheetNavigator_LastActivatedSheetName() As String
230     ILinelistSheetNavigator_LastActivatedSheetName = LastActivatedSheetName
231 End Property
232
233 Private Property Get ILinelistSheetNavigator_TranslationCacheHits() As Long
234     ILinelistSheetNavigator_TranslationCacheHits = TranslationCacheHits
235 End Property
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/LinelistSpecs.cls

```
1   VERSION 1.0 CLASS
```

```
BEGIN
  MultiUse = -1   'True
END
Attribute VB_Name = "LinelistSpecs"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = True
Attribute VB_Exposed = False


'@Folder("Linelist")
'@ModuleDescription("Modern facade exposing linelist specifications with caching and validation")
'@IgnoreModule UnrecognizedAnnotation, UseMeaningfulName, HungarianNotation

Option Explicit
Implements ILinelistSpecs

Private Type TLinelistSpecs
    specsWkb As Workbook
    dictionary As ILLdictionary
    choices As ILLChoices
    translations As ILLTranslations
    designerTranslations As IDesTranslation
    geo As ILLGeo
    exports As ILLExport
    analysis As ILLAnalysis
    passwords As IPasswords
    format As ILLFormat
    formulaData As IFormulaData
    main As IMain
    variables As ILLVariables
    translationCache As Collection
    categoryCache As Collection
    sheetCache As Collection
End Type

Private this As TLinelistSpecs

Private Const CLASSNAME As String = "LinelistSpecs"
Private Const CATEGORY_CACHE_SEPARATOR As String = "|"
Private Const DEFAULT_DICT_START_ROW As Long = 1
Private Const DEFAULT_DICT_START_COLUMN As Long = 1
Private Const DEFAULT_CHOICES_START_ROW As Long = 1
Private Const DEFAULT_CHOICES_START_COLUMN As Long = 1

'@section Factory helpers
'===============================================================================
'@description Create a new specification facade while validating workbook structure upfront.
Public Function Create(ByVal specsWorkbook As Workbook) As ILinelistSpecs
    ValidateWorkbookReference specsWorkbook

    With New LinelistSpecs
        Set .InternalWorkbook = specsWorkbook
        .ResetCaches
        Set Create = .Self
    End With
End Function

'@description Return the current instance cast as its interface for fluent construction patterns.
Public Property Get Self() As ILinelistSpecs
    Set Self = Me
End Property

'@description Primary workbook backing all specification queries.
Public Property Get Workbook() As Workbook
    EnsureWorkbook
    Set Workbook = this.specsWkb
End Property

Public Property Set InternalWorkbook(ByVal specsWorkbook As Workbook)
    Set this.specsWkb = specsWorkbook
End Property

'@section State management
'===============================================================================
'@description Reset every cached collaborator so subsequent lookups hit the workbook again.
Public Sub ResetCaches()
    ResetObjectCaches
    ResetSheetCache
    ResetTranslationCache
    ResetCategoryCache
End Sub

Private Sub ResetObjectCaches()
    Set this.dictionary = Nothing
    Set this.choices = Nothing
    Set this.translations = Nothing
```

```
89      Set this.designerTranslations = Nothing
90      Set this.geo = Nothing
91      Set this.exports = Nothing
92      Set this.analysis = Nothing
93      Set this.passwords = Nothing
94      Set this.format = Nothing
95      Set this.formulaData = Nothing
96      Set this.main = Nothing
97      Set this.variables = Nothing
98  End Sub
99
100 Private Sub ResetSheetCache()
101     Set this.sheetCache = Nothing
102 End Sub
103
104 Private Sub ResetTranslationCache()
105     Set this.translationCache = Nothing
106 End Sub
107
108 Private Sub ResetCategoryCache()
109     Set this.categoryCache = Nothing
110 End Sub
111
112 '@section Guard helpers
113 '==============================================================================
114 Private Sub EnsureWorkbook()
115     If this.specsWkb Is Nothing Then
116         ThrowError ProjectError.ObjectNotInitialized, "Specifications workbook has not been assigned"
117     End If
118 End Sub
119
120 Private Sub EnsureSheetCache()
121     If this.sheetCache Is Nothing Then Set this.sheetCache = New Collection
122 End Sub
123
124 Private Sub EnsureTranslationCache()
125     If this.translationCache Is Nothing Then Set this.translationCache = New Collection
126 End Sub
127
128 Private Sub EnsureCategoryCache()
129     If this.categoryCache Is Nothing Then Set this.categoryCache = New Collection
130 End Sub
131
132 '@section Worksheet resolution
133 '==============================================================================
134 '@description Normalise sheet names for cache keys.
135 Private Function SheetCacheKey(ByVal sheetName As String) As String
136     SheetCacheKey = LCase$(Trim$(sheetName))
137 End Function
138
139 '@description Resolve a worksheet by name with optional caching and error surfacing.
140 Private Function ResolveSheet(ByVal sheetName As String, _
141                             Optional ByVal allowMissing As Boolean = False) As Worksheet
142
143     Dim key As String
144     Dim cachedSheet As Worksheet
145     Dim target As Worksheet
146     Dim normalisedName As String
147
148     normalisedName = Trim$(sheetName)
149     If LenB(normalisedName) = 0 Then
150         If allowMissing Then Exit Function
151         ThrowError ProjectError.InvalidArgument, "Worksheet name cannot be empty"
152     End If
153
154     EnsureWorkbook
155     EnsureSheetCache
156
157     key = SheetCacheKey(normalisedName)
158
159     On Error Resume Next
160         Set cachedSheet = this.sheetCache.Item(key)
161     On Error GoTo 0
162
163     If Not (cachedSheet Is Nothing) Then
164         Set ResolveSheet = cachedSheet
165         Exit Function
166     End If
167
168     On Error Resume Next
169         Set target = this.specsWkb.Worksheets(normalisedName)
170     On Error GoTo 0
171
172     If target Is Nothing Then
173         If allowMissing Then Exit Function
174         ThrowError ProjectError.ElementNotFound, _
175                 "Worksheet '" & normalisedName & "' not found in specifications workbook '" & SafeWorkbookName(this.specsWkb) & "'"
```

```
176     End If
177
178     On Error Resume Next
179         this.sheetCache.Add target, key
180     On Error GoTo 0
181
182     Set ResolveSheet = target
183 End Function
184
185 '@section External workbook helpers
186 '==============================================================================
187 Private Sub EnterBusyState(ByRef busyStateApplied As Boolean)
188     If busyStateApplied Then Exit Sub
189
190     Application.ScreenUpdating = False
191     Application.DisplayAlerts = False
192     Application.Calculation = xlCalculationManual
193     Application.EnableAnimations = False
194
195     busyStateApplied = True
196 End Sub
197
198 Private Sub ExitBusyState(ByRef busyStateApplied As Boolean)
199     If Not busyStateApplied Then Exit Sub
200
201     Application.ScreenUpdating = True
202     Application.DisplayAlerts = True
203     Application.Calculation = xlCalculationAutomatic
204     Application.EnableAnimations = True
205
206     busyStateApplied = False
207 End Sub
208
209 Private Function OpenWorkbookInternal(ByVal workbookPath As String, _
210                                      ByRef busyStateApplied As Boolean) As Workbook
211     Dim target As Workbook
212     Dim trimmedPath As String
213
214     trimmedPath = Trim$(workbookPath)
215     If LenB(trimmedPath) = 0 Then
216         ThrowError ProjectError.ElementNotFound, _
217                 "Workbook path has not been configured"
218     End If
219
220     If LenB(Dir$(trimmedPath)) = 0 Then
221         ThrowError ProjectError.ElementNotFound, _
222                 "Workbook '" & trimmedPath & "' cannot be located"
223     End If
224
225     EnterBusyState busyStateApplied
226
227     On Error GoTo HandleOpen
228         Set target = Workbooks.Open(trimmedPath)
229     On Error GoTo 0
230
231     Set OpenWorkbookInternal = target
232     Exit Function
233
234 HandleOpen:
235     ThrowError ProjectError.ErrorUnexpectedState, _
236             "Unable to open workbook '" & trimmedPath & "': " & Err.Description
237 End Function
238
239 Private Sub CloseWorkbookQuiet(ByRef target As Workbook)
240     On Error Resume Next
241         If Not target Is Nothing Then target.Close savechanges:=False
242     On Error GoTo 0
243     Set target = Nothing
244 End Sub
245
246 Private Function ResolveExternalSheet(ByVal hostBook As Workbook, _
247                                       ByVal sheetName As String) As Worksheet
248     On Error Resume Next
249         Set ResolveExternalSheet = hostBook.Worksheets(sheetName)
250     On Error GoTo 0
251 End Function
252
253 Private Sub EnsureExternalSheetExists(ByVal hostBook As Workbook, _
254                                       ByVal sheetName As String)
255     If ResolveExternalSheet(hostBook, sheetName) Is Nothing Then
256         ThrowError ProjectError.ElementNotFound, _
257                 "Worksheet '" & sheetName & "' is missing from workbook '" & SafeWorkbookName(hostBook) & "'"
258     End If
259 End Sub
260
261 Private Sub EnsureSetupWorkbookSheets(ByVal setupBook As Workbook)
262     Dim requiredSheets As Variant
```

```
263     Dim idx As Long
264
265     requiredSheets = Array( _
266         LinelistSpecConstants.SHEET_DICTIONARY, _
267         LinelistSpecConstants.SHEET_CHOICES, _
268         LinelistSpecConstants.SHEET_TRANSLATIONS_DICT, _
269         LinelistSpecConstants.SHEET_EXPORT, _
270         LinelistSpecConstants.SHEET_ANALYSIS)
271
272     For idx = LBound(requiredSheets) To UBound(requiredSheets)
273         EnsureExternalSheetExists setupBook, CStr(requiredSheets(idx))
274     Next idx
275 End Sub
276
277 Private Sub EnsureDictionaryNotEmpty(ByVal dictionarySheet As Worksheet)
278     Dim importedDict As ILLdictionary
279     Dim dataRange As Range
280
281     Set importedDict = LLdictionary.Create(dictionarySheet, 5, 1)
282
283     On Error Resume Next
284         Set dataRange = importedDict.DataRange
285     On Error GoTo 0
286
287     If dataRange Is Nothing Then
288         ThrowError ProjectError.ElementNotFound, _
289                 "Dictionary worksheet '" & dictionarySheet.Name & "' does not expose any data"
290     End If
291
292     If Application.WorksheetFunction.CountA(dataRange) = 0 Then
293         ThrowError ProjectError.ElementNotFound, _
294                 "Dictionary worksheet '" & dictionarySheet.Name & "' is empty"
295     End If
296 End Sub
297
298 Private Function BuildPreservedSheetNames(ByVal messageTranslations As ITranslationObject) As BetterArray
299     Dim preserved As BetterArray
300
301     Set preserved = New BetterArray
302     preserved.LowerBound = 1
303
304     preserved.Push LinelistSpecConstants.SHEET_DICTIONARY, _
305                 LinelistSpecConstants.SHEET_CHOICES, _
306                 LinelistSpecConstants.SHEET_GEO, _
307                 LinelistSpecConstants.SHEET_PASSWORDS, _
308                 LinelistSpecConstants.SHEET_FORMULAS, _
309                 LinelistSpecConstants.SHEET_TRANSLATIONS_DICT, _
310                 LinelistSpecConstants.SHEET_TRANSLATIONS_LL, _
311                 LinelistSpecConstants.SHEET_EXPORT, _
312                 TemporarySheetName(LinelistSpecConstants.TEMP_SHEET_SINGLE), _
313                 TemporarySheetName(LinelistSpecConstants.TEMP_SHEET_LIST), _
314                 TemporarySheetName(LinelistSpecConstants.TEMP_SHEET_UPDATES), _
315                 TemporarySheetName(LinelistSpecConstants.TEMP_SHEET_METADATA), _
316                 TemporarySheetName(LinelistSpecConstants.TEMP_SHEET_ANALYSIS), _
317                 TemporarySheetName(LinelistSpecConstants.TEMP_SHEET_IMPORT), _
318                 TemporarySheetName(LinelistSpecConstants.TEMP_SHEET_SPATIAL), _
319                 TemporarySheetName(LinelistSpecConstants.TEMP_SHEET_SHOWHIDE)
320
321     If Not messageTranslations Is Nothing Then
322         preserved.Push SafeTranslatedValue(messageTranslations, "LLSHEET_Admin"), _
323                     SafeTranslatedValue(messageTranslations, "LLSHEET_Analysis"), _
324                     SafeTranslatedValue(messageTranslations, "LLSHEET_SpatialAnalysis"), _
325                     SafeTranslatedValue(messageTranslations, "LLSHEET_TemporalAnalysis"), _
326                     SafeTranslatedValue(messageTranslations, "LLSHEET_CustomChoice"), _
327                     SafeTranslatedValue(messageTranslations, "LLSHEET_CustomPivotTable"), _
328                     SafeTranslatedValue(messageTranslations, "LLSHEET_SpatioTemporalAnalysis")
329     End If
330
331     Set BuildPreservedSheetNames = preserved
332 End Function
333
334 Private Function SafeTranslatedValue(ByVal translations As ITranslationObject, ByVal key As String) As String
335     On Error Resume Next
336         SafeTranslatedValue = translations.TranslatedValue(key)
337     On Error GoTo 0
338
339     If LenB(SafeTranslatedValue) = 0 Then SafeTranslatedValue = key
340 End Function
341
342 Private Sub ResetPostImportCaches()
343     Set this.variables = Nothing
344     ResetCategoryCache
345     ResetTranslationCache
346 End Sub
347
348 '@section Workbook validation
349 '==========================================================================
```

```
350 Private Sub ValidateWorkbookReference(ByVal specsWorkbook As Workbook)
351     If specsWorkbook Is Nothing Then
352         ThrowError ProjectError.ObjectNotInitialized, "A specifications workbook reference is required"
353     End If
354
355     ValidateWorkbookStructure specsWorkbook
356 End Sub
357
358 Private Sub ValidateWorkbookStructure(ByVal specsWorkbook As Workbook)
359     Dim requiredSheets As Variant
360     Dim missingSheets As BetterArray
361     Dim idx As Long
362     Dim sheetName As String
363
364     requiredSheets = RequiredSheetNames()
365     Set missingSheets = New BetterArray
366     missingSheets.LowerBound = 1
367
368     For idx = LBound(requiredSheets) To UBound(requiredSheets)
369         sheetName = CStr(requiredSheets(idx))
370         If Not WorksheetExistsInternal(specsWorkbook, sheetName) Then
371             missingSheets.Push sheetName
372         End If
373     Next idx
374
375     If missingSheets.Length > 0 Then
376         ThrowError ProjectError.ElementNotFound, _
377                 "Specifications workbook '" & SafeWorkbookName(specsWorkbook) & "' is missing required worksheets: " & JoinNames(missingSheets)
378     End If
379
380     Dim formatSheet As Worksheet
381     On Error Resume Next
382         Set formatSheet = specsWorkbook.Worksheets(LinelistSpecConstants.SHEET_FORMAT)
383     On Error GoTo 0
384
385     If formatSheet Is Nothing Then Exit Sub
386
387     If Not RangeExistsInternal(formatSheet, LinelistSpecConstants.RANGE_DESIGN_TYPE) Then
388         ThrowError ProjectError.ElementNotFound, _
389                 "Worksheet '" & LinelistSpecConstants.SHEET_FORMAT & "' must define the '" & LinelistSpecConstants.RANGE_DESIGN_TYPE & "' named range"
390     End If
391 End Sub
392
393 Private Function RequiredSheetNames() As Variant
394     RequiredSheetNames = Array( _
395         LinelistSpecConstants.SHEET_DICTIONARY, _
396         LinelistSpecConstants.SHEET_CHOICES, _
397         LinelistSpecConstants.SHEET_GEO, _
398         LinelistSpecConstants.SHEET_PASSWORDS, _
399         LinelistSpecConstants.SHEET_FORMULAS, _
400         LinelistSpecConstants.SHEET_TRANSLATIONS_DICT, _
401         LinelistSpecConstants.SHEET_TRANSLATIONS_LL, _
402         LinelistSpecConstants.SHEET_ANALYSIS, _
403         LinelistSpecConstants.SHEET_EXPORT, _
404         LinelistSpecConstants.SHEET_FORMAT, _
405         LinelistSpecConstants.SHEET_MAIN, _
406         LinelistSpecConstants.SHEET_DESIGNER_TRANSLATION)
407 End Function
408
409 Private Function WorksheetExistsInternal(ByVal specsWorkbook As Workbook, _
410                                          ByVal sheetName As String) As Boolean
411     Dim target As Worksheet
412     On Error Resume Next
413         Set target = specsWorkbook.Worksheets(sheetName)
414     On Error GoTo 0
415     WorksheetExistsInternal = Not (target Is Nothing)
416 End Function
417
418 Private Function RangeExistsInternal(ByVal hostSheet As Worksheet, _
419                                      ByVal rangeName As String) As Boolean
420     Dim rng As Range
421     On Error Resume Next
422         Set rng = hostSheet.Range(rangeName)
423     On Error GoTo 0
424     RangeExistsInternal = Not (rng Is Nothing)
425 End Function
426
427 Private Function JoinNames(ByVal values As BetterArray) As String
428     Dim buffer As String
429     Dim idx As Long
430
431     For idx = values.LowerBound To values.UpperBound
432         If LenB(buffer) > 0 Then buffer = buffer & ", "
433         buffer = buffer & "'" & CStr(values.Item(idx)) & "'"
434     Next idx
435
436     JoinNames = buffer
```

```
437 End Function
438
439 Private Function SafeWorkbookName(ByVal specsWorkbook As Workbook) As String
440     On Error Resume Next
441         SafeWorkbookName = specsWorkbook.Name
442     On Error GoTo 0
443     If LenB(SafeWorkbookName) = 0 Then SafeWorkbookName = "<unknown>"
444 End Function
445
446 '@section Collaborator ensures
447 '==============================================================================
448 Private Sub EnsureExports()
449     If this.exports Is Nothing Then
450         Dim exportSheet As Worksheet
451         Set exportSheet = ResolveSheet(LinelistSpecConstants.SHEET_EXPORT)
452         Set this.exports = LLExport.Create(exportSheet)
453     End If
454 End Sub
455
456 Private Sub EnsureDictionary()
457     If this.dictionary Is Nothing Then
458         EnsureExports
459         Dim dictSheet As Worksheet
460         Set dictSheet = ResolveSheet(LinelistSpecConstants.SHEET_DICTIONARY)
461         Set this.dictionary = LLdictionary.Create(dictSheet, _
462                                                   DEFAULT_DICT_START_ROW, _
463                                                   DEFAULT_DICT_START_COLUMN, _
464                                                   NumberOfExports())
465     End If
466 End Sub
467
468 Private Sub EnsureChoices()
469     If this.choices Is Nothing Then
470         Dim choicesSheet As Worksheet
471         Set choicesSheet = ResolveSheet(LinelistSpecConstants.SHEET_CHOICES)
472         Set this.choices = LLChoices.Create(choicesSheet, _
473                                             DEFAULT_CHOICES_START_ROW, _
474                                             DEFAULT_CHOICES_START_COLUMN)
475     End If
476 End Sub
477
478 Private Sub EnsureTranslations()
479     If this.translations Is Nothing Then
480         Dim linelistTransSheet As Worksheet
481         Dim dictTransSheet As Worksheet
482         Set linelistTransSheet = ResolveSheet(LinelistSpecConstants.SHEET_TRANSLATIONS_LL)
483         Set dictTransSheet = ResolveSheet(LinelistSpecConstants.SHEET_TRANSLATIONS_DICT)
484         Set this.translations = LLTranslations.Create(linelistTransSheet, dictTransSheet)
485     End If
486 End Sub
487
488 Private Sub EnsureDesignerTranslations()
489     If this.designerTranslations Is Nothing Then
490         Dim designerSheet As Worksheet
491         Set designerSheet = ResolveSheet(LinelistSpecConstants.SHEET_DESIGNER_TRANSLATION)
492         Set this.designerTranslations = DesTranslation.Create(designerSheet)
493     End If
494 End Sub
495
496 Private Sub EnsureGeo()
497     If this.geo Is Nothing Then
498         Dim geoSheet As Worksheet
499         Set geoSheet = ResolveSheet(LinelistSpecConstants.SHEET_GEO)
500         Set this.geo = LLGeo.Create(geoSheet)
501     End If
502 End Sub
503
504 Private Sub EnsureAnalysis()
505     If this.analysis Is Nothing Then
506         Dim analysisSheet As Worksheet
507         Set analysisSheet = ResolveSheet(LinelistSpecConstants.SHEET_ANALYSIS)
508         Set this.analysis = Analysis.Create(analysisSheet)
509     End If
510 End Sub
511
512 Private Sub EnsurePasswords()
513     If this.passwords Is Nothing Then
514         Dim passwordSheet As Worksheet
515         Set passwordSheet = ResolveSheet(LinelistSpecConstants.SHEET_PASSWORDS)
516         Set this.passwords = Passwords.Create(passwordSheet)
517     End If
518 End Sub
519
520 Private Sub EnsureFormulaData()
521     If this.formulaData Is Nothing Then
522         Dim formulaSheet As Worksheet
523         Set formulaSheet = ResolveSheet(LinelistSpecConstants.SHEET_FORMULAS)
```

```
524         Set this.formulaData = FormulaData.Create(formulaSheet)
525     End If
526 End Sub
527
528 Private Sub EnsureMain()
529     If this.main Is Nothing Then
530         Dim mainSheet As Worksheet
531         Set mainSheet = ResolveSheet(LinelistSpecConstants.SHEET_MAIN)
532         Set this.main = Main.Create(mainSheet)
533     End If
534 End Sub
535
536 Private Sub EnsureVariables()
537     EnsureDictionary
538     If this.variables Is Nothing Then
539         Set this.variables = LLVariables.Create(this.dictionary)
540     End If
541 End Sub
542
543 Private Sub EnsureFormat()
544     If this.format Is Nothing Then
545         Dim formatSheet As Worksheet
546         Dim designName As String
547
548         Set formatSheet = ResolveSheet(LinelistSpecConstants.SHEET_FORMAT)
549         designName = ResolveDesignName(formatSheet)
550
551         Set this.format = LLFormat.Create(formatSheet, designName)
552     End If
553 End Sub
554
555 Private Function ResolveDesignName(ByVal formatSheet As Worksheet) As String
556     Dim designRange As Range
557
558     On Error Resume Next
559         Set designRange = formatSheet.Range(LinelistSpecConstants.RANGE_DESIGN_TYPE)
560     On Error GoTo 0
561
562     If designRange Is Nothing Then
563         ThrowError ProjectError.ElementNotFound, _
564                 "Named range '" & LinelistSpecConstants.RANGE_DESIGN_TYPE & "' missing from format worksheet"
565     End If
566
567     ResolveDesignName = Trim$(CStr(designRange.Cells(1, 1).Value))
568 End Function
569
570 Private Sub EnsureTranslationObjectCache()
571     EnsureTranslations
572     EnsureTranslationCache
573 End Sub
574
575 Private Sub EnsureCategoryLookup()
576     EnsureChoices
577     EnsureVariables
578     EnsureCategoryCache
579 End Sub
580
581 Private Function NumberOfExports() As Long
582     EnsureExports
583     NumberOfExports = this.exports.NumberOfExports()
584 End Function
585
586 '@section List auto helpers
587 '==============================================================================
588 Private Sub AddListAuto()
589     Dim dict As ILLdictionary
590     Dim vars As ILLVariables
591     Dim listVars As BetterArray
592     Dim idx As Long
593     Dim varName As String
594     Dim controlType As String
595
596     Set dict = Dictionary()
597     EnsureVariables
598     Set vars = this.variables
599
600     dict.AddColumn "list auto"
601     Set listVars = dict.Data.FilterData("control", "list_auto", "control details")
602
603     If listVars Is Nothing Then Exit Sub
604
605     For idx = listVars.LowerBound To listVars.UpperBound
606         varName = CStr(listVars.Item(idx))
607
608         vars.SetValue varName:=varName, colName:="list auto", newValue:="list_auto_origin", onEmpty:=True
609
610         controlType = LCase$(Trim$(vars.Value(varName:=varName, colName:="control")))
```

```
611         Select Case controlType
612             Case "formula", "case_when", "choice_formula"
613                 RecursiveListAuto varName
614         End Select
615     Next idx
616 End Sub
617
618 Private Sub RecursiveListAuto(ByVal varName As String)
619     Dim dict As ILLdictionary
620     Dim vars As ILLVariables
621     Dim controlType As String
622     Dim controlDetails As String
623     Dim formulaEngine As IFormulas
624     Dim referencedVars As BetterArray
625     Dim idx As Long
626     Dim referencedName As String
627     Dim referencedControl As String
628
629     Set dict = Dictionary()
630     EnsureVariables
631     Set vars = this.variables
632
633     controlType = LCase$(Trim$(vars.Value(varName:=varName, colName:="control")))
634     If controlType <> "case_when" And controlType <> "formula" And controlType <> "choice_formula" Then Exit Sub
635
636     controlDetails = Trim$(vars.Value(varName:=varName, colName:="control details"))
637     If LenB(controlDetails) = 0 Then Exit Sub
638
639     Set formulaEngine = Formulas.Create(dict, FormulaDataObject(), controlDetails)
640     If Not formulaEngine.Valid("linelist") Then Exit Sub
641
642     Set referencedVars = formulaEngine.Varlists("linelist")
643     If referencedVars Is Nothing Then Exit Sub
644
645     For idx = referencedVars.LowerBound To referencedVars.UpperBound
646         referencedName = CStr(referencedVars.Item(idx))
647
648         vars.SetValue varName:=referencedName, colName:="list auto", newValue:="list_auto_origin", onEmpty:=True
649
650         referencedControl = LCase$(Trim$(vars.Value(varName:=referencedName, colName:="control")))
651         If referencedControl = "formula" Or referencedControl = "case_when" Then
652             RecursiveListAuto referencedName
653         End If
654     Next idx
655 End Sub
656
657 '@section Category helpers
658 '==============================================================================
659 Public Function Categories(ByVal varName As String, Optional ByVal useShortlabels As Boolean = False) As BetterArray
660     Dim cacheKey As String
661     Dim cached As BetterArray
662     Dim clone As BetterArray
663
664     EnsureCategoryLookup
665
666     cacheKey = BuildCategoryCacheKey(varName, useShortlabels)
667
668     On Error Resume Next
669         Set cached = this.categoryCache.Item(cacheKey)
670     On Error GoTo 0
671
672     If cached Is Nothing Then
673         Set cached = ResolveCategoryList(varName, useShortlabels)
674         On Error Resume Next
675             this.categoryCache.Add cached, cacheKey
676         On Error GoTo 0
677     End If
678
679     Set clone = cached.Clone
680     Set Categories = clone
681 End Function
682
683 Private Function ResolveCategoryList(ByVal varName As String, _
684                                     ByVal useShortlabels As Boolean) As BetterArray
685     Dim normalizedName As String
686     Dim controlValue As String
687     Dim controlDetails As String
688     Dim categoryName As String
689     Dim result As BetterArray
690
691     normalizedName = Trim$(varName)
692     If LenB(normalizedName) = 0 Then
693         ThrowError ProjectError.InvalidArgument, "Variable name is required to resolve categories"
694     End If
695
696     controlValue = NormaliseControlType(this.variables.Value(varName:=normalizedName, colName:="control"))
697     controlDetails = Trim$(this.variables.Value(varName:=normalizedName, colName:="control details"))
```

```
    Select Case controlValue
        Case "choice_formula"
            categoryName = ExtractChoiceFormula(controlDetails)
        Case "choice_manual", "choice_multiple", "choice_custom"
            categoryName = controlDetails
        Case Else
            ThrowError ProjectError.ElementNotFound, _
                    "Variable '" & normalizedName & "' does not expose selectable categories"
    End Select

    categoryName = Trim$(categoryName)
    If LenB(categoryName) = 0 Then
        ThrowError ProjectError.ElementNotFound, _
                "Variable '" & normalizedName & "' references an empty category definition"
    End If

    Set result = this.choices.Categories(categoryName, useShortlabels)
    If result Is Nothing Then
        ThrowError ProjectError.ElementNotFound, _
                "Categories for '" & categoryName & "' are missing from the choices sheet"
    End If

    Set ResolveCategoryList = result
End Function

Private Function BuildCategoryCacheKey(ByVal varName As String, _
                                       ByVal useShortlabels As Boolean) As String
    BuildCategoryCacheKey = LCase$(Trim$(varName)) & CATEGORY_CACHE_SEPARATOR & CStr(useShortlabels)
End Function

Private Function NormaliseControlType(ByVal controlValue As String) As String
    Dim trimmed As String
    Dim delimiterPos As Long

    trimmed = LCase$(Trim$(controlValue))
    delimiterPos = InStr(1, trimmed, "(", vbBinaryCompare)
    If delimiterPos > 0 Then
        trimmed = Left$(trimmed, delimiterPos - 1)
    End If

    NormaliseControlType = trimmed
End Function

Private Function ExtractChoiceFormula(ByVal controlDetails As String) As String
    Dim working As String
    Dim commaPosition As Long

    working = controlDetails
    working = Replace$(working, "CHOICE_FORMULA(", vbNullString, 1, -1, vbTextCompare)

    commaPosition = InStr(1, working, ",", vbBinaryCompare)
    If commaPosition > 0 Then
        working = Left$(working, commaPosition - 1)
    End If

    working = Replace$(working, ")", vbNullString, 1, -1, vbBinaryCompare)
    ExtractChoiceFormula = Trim$(working)
End Function

'@section Core accessors
'==============================================================================
Public Property Get Dictionary() As ILLdictionary
    EnsureDictionary
    Set Dictionary = this.dictionary
End Property

Public Property Get Choices() As ILLChoices
    EnsureChoices
    Set Choices = this.choices
End Property

Public Property Get DesignFormat() As ILLFormat
    EnsureFormat
    Set DesignFormat = this.format
End Property

Public Property Get Analysis() As ILLAnalysis
    EnsureAnalysis
    Set Analysis = this.analysis
End Property

Public Property Get ExportObject() As ILLExport
    EnsureExports
    Set ExportObject = this.exports
End Property
```

```
785 Public Property Get Password() As IPasswords
786    EnsurePasswords
787    Set Password = this.passwords
788 End Property
789
790 Public Property Get GeoObject() As ILLGeo
791    EnsureGeo
792    Set GeoObject = this.geo
793 End Property
794
795 Public Property Get FormulaDataObject() As IFormulaData
796    EnsureFormulaData
797    Set FormulaDataObject = this.formulaData
798 End Property
799
800 Public Property Get MainObject() As IMain
801    EnsureMain
802    Set MainObject = this.main
803 End Property
804
805 Public Property Get Translations() As ILLTranslations
806    EnsureTranslations
807    Set Translations = this.translations
808 End Property
809
810 Public Property Get DesignerTranslations() As IDesTranslation
811    EnsureDesignerTranslations
812    Set DesignerTranslations = this.designerTranslations
813 End Property
814
815 Public Property Get TransObject(Optional ByVal translationScope As Byte = TranslationOfMessages) As ITranslationObject
816    Dim cacheKey As String
817    Dim cached As ITranslationObject
818    Dim resolved As ITranslationObject
819
820    EnsureTranslationObjectCache
821
822    cacheKey = CStr(translationScope)
823
824    On Error Resume Next
825       Set cached = this.translationCache.Item(cacheKey)
826    On Error GoTo 0
827
828    If cached Is Nothing Then
829       Set resolved = this.translations.TransObject(translationScope)
830       If resolved Is Nothing Then
831          ThrowError ProjectError.ElementNotFound, _
832                "Translation scope '" & CStr(translationScope) & "' is not available in the specifications workbook"
833       End If
834       On Error Resume Next
835          this.translationCache.Add resolved, cacheKey
836       On Error GoTo 0
837       Set cached = resolved
838    End If
839
840    Set TransObject = cached
841 End Property
842
843 Public Property Get TemporarySheetName(ByVal scope As Byte) As String
844    Select Case scope
845       Case LinelistSpecConstants.TEMP_SHEET_ANALYSIS
846          TemporarySheetName = "ana_tabnames__"
847       Case LinelistSpecConstants.TEMP_SHEET_UPDATES
848          TemporarySheetName = "updates__"
849       Case LinelistSpecConstants.TEMP_SHEET_METADATA
850          TemporarySheetName = "Metadata"
851       Case LinelistSpecConstants.TEMP_SHEET_LIST
852          TemporarySheetName = "dropdown_lists__"
853       Case LinelistSpecConstants.TEMP_SHEET_IMPORT
854          TemporarySheetName = "import_rep__"
855       Case LinelistSpecConstants.TEMP_SHEET_SINGLE
856          TemporarySheetName = "temp__"
857       Case LinelistSpecConstants.TEMP_SHEET_SPATIAL
858          TemporarySheetName = "spatial_tables__"
859       Case LinelistSpecConstants.TEMP_SHEET_SHOWHIDE
860          TemporarySheetName = "show_hide__"
861       Case Else
862          ThrowError ProjectError.InvalidArgument, _
863                "Unknown temporary sheet scope '" & CStr(scope) & "'"
864    End Select
865 End Property
866
867 Public Property Get Value(ByVal tagName As String) As String
868    Dim normalizedTag As String
869
870    normalizedTag = LCase$(Trim$(tagName))
871    If LenB(normalizedTag) = 0 Then Exit Property
```

```
872
873     Select Case normalizedTag
874         Case "numberofexports"
875             Value = CStr(NumberOfExports())
876         Case Else
877             EnsureMain
878             Value = this.main.ValueOf(tagName)
879     End Select
880 End Property
881
882 '@section Writer factories
883 '==============================================================================
884 Public Function CreateSectionContext(ByVal linelist As ILinelist, ByVal startRow As Long) As ILLSectionContext
885     If linelist Is Nothing Then
886         ThrowError ProjectError.ObjectNotInitialized, "Linelist reference is required to build a section context"
887     End If
888     If startRow <= 0 Then
889         ThrowError ProjectError.InvalidArgument, "Section context start row must be greater than zero"
890     End If
891
892     Dim context As ILLSectionContext
893     Set context = New LLSectionContext
894     context.Initialise linelist, startRow
895     Set CreateSectionContext = context
896 End Function
897
898 Public Function CreateSectionWriter() As ILLSectionWriter
899     Dim writer As ILLSectionWriter
900     Set writer = New LLSectionWriterBase
901     Set CreateSectionWriter = writer
902 End Function
903
904 Public Function CreateSubSectionWriter() As ILLSubSectionWriter
905     Dim writer As ILLSubSectionWriter
906     Set writer = New LLSubSectionWriterBase
907     Set CreateSubSectionWriter = writer
908 End Function
909
910 Public Function CreateVarContext(ByVal linelist As ILinelist, ByVal variableName As String) As ILLVarContext
911     Dim trimmedName As String
912
913     If linelist Is Nothing Then
914         ThrowError ProjectError.ObjectNotInitialized, "Linelist reference is required to build a variable context"
915     End If
916
917     trimmedName = Trim$(variableName)
918     If LenB(trimmedName) = 0 Then
919         ThrowError ProjectError.InvalidArgument, "Variable name is required to create a variable context"
920     End If
921
922     Set CreateVarContext = LLVarContext.Create(linelist, trimmedName)
923 End Function
924
925 Public Function CreateVarWriter() As ILLVarWriter
926     Dim writer As ILLVarWriter
927     Set writer = New LLVarWriterBase
928     Set CreateVarWriter = writer
929 End Function
930
931 '@section Workbook orchestration
932 '==============================================================================
933 Public Sub Prepare()
934     Dim mainObj As IMain
935     Dim designerTrads As IDesTranslation
936     Dim setupPath As String
937     Dim geoPath As String
938     Dim dict As ILLdictionary
939     Dim choices As ILLChoices
940     Dim exports As ILLExport
941     Dim translations As ILLTranslations
942     Dim analysisObj As ILLAnalysis
943     Dim geoObj As ILLGeo
944     Dim dictTrads As ITranslationObject
945     Dim msgTrads As ITranslationObject
946     Dim preservedNames As BetterArray
947     Dim setupBook As Workbook
948     Dim geoBook As Workbook
949     Dim busyStateApplied As Boolean
950     Dim prepared As Boolean
951     Dim dictionarySheet As Worksheet
952     Dim choicesSheet As Worksheet
953     Dim analysisSheet As Worksheet
954
955     On Error GoTo CleanFail
956
957     Set mainObj = MainObject()
958     Set designerTrads = DesignerTranslations()
```

```
959
960     mainObj.CheckReadiness designerTrads
961     If Not mainObj.Ready Then GoTo CleanExit
962
963     setupPath = Trim$(mainObj.ValueOf("setuppath"))
964     If LenB(setupPath) = 0 Then
965         ThrowError ProjectError.ElementNotFound, _
966                 "Setup workbook path is not configured on the Main sheet"
967     End If
968
969     geoPath = Trim$(mainObj.ValueOf("geopath"))
970
971     Set dict = Dictionary()
972     Set choices = Choices()
973     Set exports = ExportObject()
974     Set translations = Translations()
975     Set analysisObj = Analysis()
976     Set geoObj = GeoObject()
977
978     Set setupBook = OpenWorkbookInternal(setupPath, busyStateApplied)
979     EnsureSetupWorkbookSheets setupBook
980
981     exports.ImportSpecs setupBook.Worksheets(LinelistSpecConstants.SHEET_EXPORT), 4, 1
982
983     Set dictionarySheet = setupBook.Worksheets(LinelistSpecConstants.SHEET_DICTIONARY)
984     EnsureDictionaryNotEmpty dictionarySheet
985     dict.Import dictionarySheet, fromStartRow:=5, fromStartcol:=1
986
987     Set choicesSheet = setupBook.Worksheets(LinelistSpecConstants.SHEET_CHOICES)
988     choices.Import choicesSheet, fromStartRow:=4, fromStartcol:=1
989
990     translations.ImportTranslation setupBook
991
992     Set analysisSheet = setupBook.Worksheets(LinelistSpecConstants.SHEET_ANALYSIS)
993     analysisObj.Import analysisSheet
994
995     CloseWorkbookQuiet setupBook
996
997     If LenB(geoPath) > 0 Then
998         Set geoBook = OpenWorkbookInternal(geoPath, busyStateApplied)
999         geoObj.Import geoBook
1000        CloseWorkbookQuiet geoBook
1001    End If
1002
1003    Set dictTrads = TransObject(TranslationOfDictionary)
1004    Set msgTrads = TransObject()
1005
1006    Set preservedNames = BuildPreservedSheetNames(msgTrads)
1007    dict.Prepare preservedNames, geoObj, msgTrads, "table"
1008
1009    If Not dict.ColumnExists("list auto") Then
1010        AddListAuto
1011    End If
1012
1013    dict.Translate dictTrads
1014    choices.Translate dictTrads
1015    choices.Sort
1016    exports.Translate dictTrads
1017    analysisObj.Translate dictTrads
1018
1019    prepared = True
1020
1021 CleanExit:
1022    CloseWorkbookQuiet setupBook
1023    CloseWorkbookQuiet geoBook
1024    ExitBusyState busyStateApplied
1025    If prepared Then ResetPostImportCaches
1026    Exit Sub
1027
1028 CleanFail:
1029    Dim errNumber As Long
1030    Dim errDescription As String
1031
1032    errNumber = Err.Number
1033    errDescription = Err.Description
1034
1035    CloseWorkbookQuiet setupBook
1036    CloseWorkbookQuiet geoBook
1037    ExitBusyState busyStateApplied
1038
1039    If errNumber = 0 Then Exit Sub
1040
1041    If errNumber >= ProjectError.ErrorNone And errNumber <= ProjectError.ElementNotFound Then
1042        Err.Raise errNumber, CLASSNAME, errDescription
1043    Else
1044        ThrowError ProjectError.ErrorUnexpectedState, errDescription
1045    End If
```

```
1046 End Sub
1047
1048 Public Sub Export(ByVal toWkb As Workbook)
1049    Dim dict As ILLdictionary
1050    Dim choices As ILLChoices
1051    Dim translations As ILLTranslations
1052    Dim exports As ILLExport
1053    Dim geo As ILLGeo
1054    Dim passwords As IPasswords
1055    Dim mainObj As IMain
1056
1057    On Error GoTo HandleError
1058
1059    If toWkb Is Nothing Then
1060        ThrowError ProjectError.ObjectNotInitialized, _
1061                "Destination workbook is required to export specifications"
1062    End If
1063
1064    Set dict = Dictionary()
1065    Set choices = Choices()
1066    Set translations = Translations()
1067    Set exports = ExportObject()
1068    Set geo = GeoObject()
1069    Set passwords = Password()
1070    Set mainObj = MainObject()
1071
1072    dict.Export toWkb, Hide:=xlSheetVeryHidden
1073    choices.Export toWkb, Hide:=xlSheetVeryHidden
1074    translations.ExportToWkb toWkb
1075    exports.ExportSpecs toWkb, Hide:=xlSheetVeryHidden
1076    geo.ExportToWkb toWkb, mainObj.ValueOf("llname"), mainObj.ValueOf("setuppath")
1077    passwords.Export toWkb
1078
1079    Exit Sub
1080
1081 HandleError:
1082    Dim errNumber As Long
1083    Dim errDescription As String
1084
1085    errNumber = Err.Number
1086    errDescription = Err.Description
1087
1088    If errNumber = 0 Then Exit Sub
1089
1090    If errNumber >= ProjectError.ErrorNone And errNumber <= ProjectError.ElementNotFound Then
1091        Err.Raise errNumber, CLASSNAME, errDescription
1092    Else
1093        ThrowError ProjectError.ErrorUnexpectedState, errDescription
1094    End If
1095 End Sub
1096
1097 Public Sub ErrorManage(Optional ByVal textMessage As String = vbNullString)
1098    Dim messageBody As String
1099
1100    messageBody = "An error occurred while building the linelist specifications." & vbNewLine & _
1101                "Details: " & textMessage & vbNewLine & _
1102                "Aborting linelist creation process."
1103
1104    MsgBox messageBody, vbExclamation + vbOKOnly, "Linelist Specifications"
1105 End Sub
1106
1107 '@section Error handling
1108 '==============================================================================
1109 Private Sub ThrowError(ByVal errNumber As Long, ByVal messageText As String)
1110    Err.Raise errNumber, CLASSNAME, messageText
1111 End Sub
1112
1113 '@section Test support
1114 '==============================================================================
1115 Public Sub TestAssignMain(ByVal mainObj As IMain)
1116    Set this.main = mainObj
1117 End Sub
1118
1119 Public Sub TestAssignDesignerTranslations(ByVal designerTrads As IDesTranslation)
1120    Set this.designerTranslations = designerTrads
1121 End Sub
1122
1123 Public Sub TestAssignTranslations(ByVal translations As ILLTranslations)
1124    Set this.translations = translations
1125    ResetTranslationCache
1126 End Sub
1127
1128 '@section Interface implementation
1129 '==============================================================================
1130 Private Sub ILinelistSpecs_ResetCaches()
1131    ResetCaches
1132 End Sub
```

```
1133
1134 Private Property Get ILinelistSpecs_Workbook() As Workbook
1135    Set ILinelistSpecs_Workbook = Me.Workbook
1136 End Property
1137
1138 Private Property Get ILinelistSpecs_Dictionary() As ILLdictionary
1139    Set ILinelistSpecs_Dictionary = Me.Dictionary
1140 End Property
1141
1142 Private Property Get ILinelistSpecs_Choices() As ILLChoices
1143    Set ILinelistSpecs_Choices = Me.Choices
1144 End Property
1145
1146 Private Property Get ILinelistSpecs_DesignFormat() As ILLFormat
1147    Set ILinelistSpecs_DesignFormat = Me.DesignFormat
1148 End Property
1149
1150 Private Property Get ILinelistSpecs_Analysis() As ILLAnalysis
1151    Set ILinelistSpecs_Analysis = Me.Analysis
1152 End Property
1153
1154 Private Property Get ILinelistSpecs_ExportObject() As ILLExport
1155    Set ILinelistSpecs_ExportObject = Me.ExportObject
1156 End Property
1157
1158 Private Property Get ILinelistSpecs_Password() As IPasswords
1159    Set ILinelistSpecs_Password = Me.Password
1160 End Property
1161
1162 Private Property Get ILinelistSpecs_GeoObject() As ILLGeo
1163    Set ILinelistSpecs_GeoObject = Me.GeoObject
1164 End Property
1165
1166 Private Property Get ILinelistSpecs_FormulaDataObject() As IFormulaData
1167    Set ILinelistSpecs_FormulaDataObject = Me.FormulaDataObject
1168 End Property
1169
1170 Private Property Get ILinelistSpecs_MainObject() As IMain
1171    Set ILinelistSpecs_MainObject = Me.MainObject
1172 End Property
1173
1174 Private Property Get ILinelistSpecs_Translations() As ILLTranslations
1175    Set ILinelistSpecs_Translations = Me.Translations
1176 End Property
1177
1178 Private Property Get ILinelistSpecs_DesignerTranslations() As IDesTranslation
1179    Set ILinelistSpecs_DesignerTranslations = Me.DesignerTranslations
1180 End Property
1181
1182 Private Property Get ILinelistSpecs_TransObject(Optional ByVal translationScope As Byte = TranslationOfMessages) As ITranslationObject
1183    Set ILinelistSpecs_TransObject = Me.TransObject(translationScope)
1184 End Property
1185
1186 Private Property Get ILinelistSpecs_TemporarySheetName(ByVal scope As Byte) As String
1187    ILinelistSpecs_TemporarySheetName = Me.TemporarySheetName(scope)
1188 End Property
1189
1190 Private Property Get ILinelistSpecs_Value(ByVal tagName As String) As String
1191    ILinelistSpecs_Value = Me.Value(tagName)
1192 End Property
1193
1194 Private Function ILinelistSpecs_Categories(ByVal varName As String, Optional ByVal useShortlabels As Boolean = False) As BetterArray
1195    Set ILinelistSpecs_Categories = Me.Categories(varName, useShortlabels)
1196 End Function
1197
1198 Private Sub ILinelistSpecs_Prepare()
1199    Prepare
1200 End Sub
1201
1202 Private Sub ILinelistSpecs_Export(ByVal toWkb As Workbook)
1203    Export toWkb
1204 End Sub
1205
1206 Private Sub ILinelistSpecs_ErrorManage(Optional ByVal textMessage As String = vbNullString)
1207    ErrorManage textMessage
1208 End Sub
1209
1210 Private Function ILinelistSpecs_CreateSectionContext(ByVal linelist As ILinelist, ByVal startRow As Long) As ILLSectionContext
1211    Set ILinelistSpecs_CreateSectionContext = CreateSectionContext(linelist, startRow)
1212 End Function
1213
1214 Private Function ILinelistSpecs_CreateSectionWriter() As ILLSectionWriter
1215    Set ILinelistSpecs_CreateSectionWriter = CreateSectionWriter()
1216 End Function
1217
1218 Private Function ILinelistSpecs_CreateSubSectionWriter() As ILLSubSectionWriter
1219    Set ILinelistSpecs_CreateSubSectionWriter = CreateSubSectionWriter()
```

```
End Function

Private Function ILinelistSpecs_CreateVarContext(ByVal linelist As ILinelist, ByVal variableName As String) As ILLVarContext
    Set ILinelistSpecs_CreateVarContext = CreateVarContext(linelist, variableName)
End Function

Private Function ILinelistSpecs_CreateVarWriter() As ILLVarWriter
    Set ILinelistSpecs_CreateVarWriter = CreateVarWriter()
End Function
```

/Users/komlaviamevoin/Unsync-Working-Folders/outbreak-tools/src/classes/linelist/LinelistVBIDECodeTransferStrategy.cls

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1   'True
END
Attribute VB_Name = "LinelistVBIDECodeTransferStrategy"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Attribute VB_Description = "Transfers VBA components using VBIDE import/export routines"

'@Folder("Linelist")
'@ModuleDescription("Transfers VBA components using VBIDE import/export routines")
'@IgnoreModule UnrecognizedAnnotation, UseMeaningfulName, ProcedureNotUsed
'@depends ProjectError

Option Explicit

Implements ILinelistCodeTransferStrategy

Private Const CLASSNAME As String = "LinelistVBIDECodeTransferStrategy"
Private Const TEMPFILEPREFIX As String = "LLTransfer_"


'@section Factory helpers
'==============================================================================
Public Function Create() As ILinelistCodeTransferStrategy
    Dim strategy As LinelistVBIDECodeTransferStrategy
    Set strategy = New LinelistVBIDECodeTransferStrategy
    Set Create = strategy
End Function


'@section Strategy contract
'==============================================================================
Public Sub TransferComponent(ByVal componentName As String, _
                             ByVal scope As LinelistCodeScope, _
                             ByVal source As Workbook, _
                             ByVal target As Workbook)
    Dim extension As String
    Dim exportPath As String
    Dim sourceComponent As Object
    Dim targetComponent As Object
    Dim targetProject As Object

    extension = FileExtension(scope)
    exportPath = BuildTempPath(extension, source)

    Set sourceComponent = ResolveComponent(source, componentName)

    On Error GoTo CleanUp
        sourceComponent.Export exportPath
        Set targetProject = target.VBProject
        If ComponentExists(targetProject, componentName) Then
            targetProject.VBComponents.Remove targetProject.VBComponents(componentName)
        End If
        targetProject.VBComponents.Import exportPath
    On Error GoTo 0

CleanUp:
    SafeKill exportPath
    If Err.Number <> 0 Then
        ThrowError ProjectError.ErrorUnexpectedState, _
                "Failed to transfer component '" & componentName & "' using VBIDE: " & Err.Description
    End If
End Sub
```

```
67
68  Public Sub TransferWorkbookModule(ByVal moduleName As String, _
69                                    ByVal source As Workbook, _
70                                    ByVal target As Workbook)
71      Dim sourceModule As Object
72      Dim targetModule As Object
73      Dim moduleCode As String
74
75      Set sourceModule = ResolveComponent(source, moduleName).CodeModule
76      Set targetModule = ResolveComponent(target, moduleName).CodeModule
77
78      moduleCode = sourceModule.Lines(1, sourceModule.CountOfLines)
79
80      With targetModule
81          If .CountOfLines > 0 Then .DeleteLines 1, .CountOfLines
82          If Len(moduleCode) > 0 Then .AddFromString moduleCode
83      End With
84  End Sub
85
86  Public Sub TransferForm(ByVal formName As String, _
87                          ByVal source As Workbook, _
88                          ByVal target As Workbook)
89      Dim exportPath As String
90      Dim sourceComponent As Object
91      Dim targetProject As Object
92
93      exportPath = BuildTempPath(".frm", source)
94      Set sourceComponent = ResolveComponent(source, formName)
95
96      On Error GoTo CleanUp
97          sourceComponent.Export exportPath
98          Set targetProject = target.VBProject
99          If ComponentExists(targetProject, formName) Then
100             targetProject.VBComponents.Remove targetProject.VBComponents(formName)
101         End If
102         targetProject.VBComponents.Import exportPath
103     On Error GoTo 0
104
105 CleanUp:
106     SafeKill exportPath
107     SafeKill Replace$(exportPath, ".frm", ".frx")
108     If Err.Number <> 0 Then
109         ThrowError ProjectError.ErrorUnexpectedState, _
110                 "Failed to transfer form '" & formName & "' using VBIDE: " & Err.Description
111     End If
112 End Sub
113
114 Public Property Get Name() As String
115     Name = "VBIDE"
116 End Property
117
118
119 '@section Helpers
120 '===============================================================================
121 Private Function ResolveComponent(ByVal workbook As Workbook, ByVal componentName As String) As Object
122     Dim vbProject As Object
123
124     On Error GoTo Missing
125         Set vbProject = workbook.VBProject
126         Set ResolveComponent = vbProject.VBComponents(componentName)
127     On Error GoTo 0
128     Exit Function
129
130 Missing:
131     ThrowError ProjectError.ElementNotFound, _
132             "Component '" & componentName & "' not found in workbook '" & workbook.Name & "'"
133 End Function
134
135 Private Function ComponentExists(ByVal project As Object, ByVal componentName As String) As Boolean
136     On Error GoTo Missing
137         project.VBComponents componentName
138         ComponentExists = True
139         Exit Function
140 Missing:
141     Err.Clear
142     ComponentExists = False
143 End Function
144
145 Private Function TempDirectory(Optional ByVal referenceBook As Workbook) As String
146     Dim candidate As String
147
148     On Error Resume Next
149         If Not referenceBook Is Nothing Then
150             candidate = referenceBook.Path
151         End If
152     On Error GoTo 0
153
```

```
154     If LenB(candidate) = 0 Then
155         On Error Resume Next
156             candidate = Application.Path
157         On Error GoTo 0
158     End If
159
160     If LenB(candidate) = 0 Then
161         candidate = CurDir$
162     End If
163
164     If LenB(candidate) = 0 Then
165         ThrowError ProjectError.ErrorUnexpectedState, "Unable to determine a temporary directory for exports"
166     End If
167
168     If Right$(candidate, 1) <> Application.PathSeparator Then
169         candidate = candidate & Application.PathSeparator
170     End If
171
172     TempDirectory = candidate
173 End Function
174
175 Private Function BuildTempPath(ByVal extension As String, _
176                               Optional ByVal referenceBook As Workbook) As String
177     Dim folder As String
178     Dim fileName As String
179
180     Randomize
181     folder = TempDirectory(referenceBook)
182     fileName = TEMPFILEPREFIX & Format$(Now, "yyyymmdd_hhnnss") & "_" & CLng(Rnd * 100000)
183     BuildTempPath = folder & fileName & extension
184 End Function
185
186 Private Function FileExtension(ByVal scope As LinelistCodeScope) As String
187     Select Case scope
188         Case codeScopeClass
189             FileExtension = ".cls"
190         Case codeScopeModule
191             FileExtension = ".bas"
192         Case Else
193             ThrowError ProjectError.InvalidArgument, "Unsupported code scope value " & CStr(scope)
194     End Select
195 End Function
196
197 Private Sub SafeKill(ByVal path As String)
198     If LenB(path) = 0 Then Exit Sub
199     On Error Resume Next
200         Kill path
201     On Error GoTo 0
202 End Sub
203
204 Private Sub ThrowError(ByVal errNumber As ProjectError, ByVal message As String)
205     Err.Raise errNumber, CLASSNAME, message
206 End Sub
207
208
209 '@section Interface implementation
210 '==============================================================================
211 Private Sub ILinelistCodeTransferStrategy_TransferComponent(ByVal componentName As String, _
212                                                            ByVal scope As LinelistCodeScope, _
213                                                            ByVal source As Workbook, _
214                                                            ByVal target As Workbook)
215     TransferComponent componentName, scope, source, target
216 End Sub
217
218 Private Sub ILinelistCodeTransferStrategy_TransferWorkbookModule(ByVal moduleName As String, _
219                                                                  ByVal source As Workbook, _
220                                                                  ByVal target As Workbook)
221     TransferWorkbookModule moduleName, source, target
222 End Sub
223
224 Private Sub ILinelistCodeTransferStrategy_TransferForm(ByVal formName As String, _
225                                                        ByVal source As Workbook, _
226                                                        ByVal target As Workbook)
227     TransferForm formName, source, target
228 End Sub
229
230 Private Property Get ILinelistCodeTransferStrategy_Name() As String
231     ILinelistCodeTransferStrategy_Name = Name
232 End Property
```